



Thirsty-Lang and the UTF

Universal Thirsty Family

A Complete Technical, Architectural, and Governance Reference

Version: 1.2

Date: 2026-05-12

Repository: IAmSoThirsty/Project-AI

Status: Authoritative Reference — Compiled from Repository Sources

License: MIT

Thirsty-Lang is the human-readable orchestration layer for governance-bound execution inside Project-AI. The UTF — Universal Thirsty Family — is the broader family of languages, encodings, runtimes, and symbolic systems that carry Project-AI's constitutional execution model across source code, policy, bytecode, simulation, and runtime enforcement.

UTF Implementation Statistics

Metric	Value
Total Python Files	41
Total Lines of Code	5,995
Total Size	246.73 KB
Example Files	7
Test Coverage	7 test files, 362 test lines
Subdirectories	19

Module Breakdown

Module	Files	Lines	Description
Thirsty-Lang (Tier 1)	17	4,335	Lexer, parser, interpreter, CLI
T.A.R.L. (Tier 3)	6	753	Policy runtime and bytecode VM
Shadow Thirst (Tier 4)	3	262	Mutation simulation layer
TSCG (Tier 5)	3	139	Symbolic constitutional grammar
TSCG-B (Tier 6)	3	135	Binary frame protocol
Test Suite	8	362	Unit and integration tests

UTF Feature Matrix — Six-Tier Architecture

The Universal Thirsty Family implements a governance-first execution model across six integrated tiers, each building upon the previous layer.

Tier	Layer	Purpose	Key Features	Status
1	Thirsty-Lang	Source syntax	Water metaphor keywords, type system	✓ Implemented
2	Thirst of Gods	Advanced dialect	Extended stdlib, async/await patterns	✓ Implemented
3	T.A.R.L.	Policy/Runtime	Governance policies, bytecode VM	✓ Implemented
4	Shadow Thirst	Mutation sim	Shadow/invariant/canonical execution	✓ Implemented
5	TSCG	Symbolic grammar	9 symbols (COG,DNT,SHD,INV,CAP,QRM,COMMAND,REF)	COMPLETED
6	TSCG-B	Binary protocol	Frame-based, CRC32+SHA256 integrity	✓ Implemented

Architecture Overview — Component Directory

UTF is implemented as a cohesive Python package under `src/utf/` with clear separation of concerns across lexing, parsing, interpretation, policy enforcement, mutation validation, and serialization.

Component	Location	Files	Lines	Role
Lexer	<code>src/utf/thirsty_lang/lexer.py</code>	1	~500	Tokenization
Parser	<code>src/utf/thirsty_lang/parser.py</code>	1	~800	AST generation
Interpreter	<code>src/utf/thirsty_lang/interpreter.py</code>	1	~1200	Execution engine
CLI	<code>src/utf/thirsty_lang/cli.py</code>	1	~300	Command-line interface
TARL Runtime	<code>src/utf/tarl/</code>	6	753	Policy enforcement
Shadow Thirst	<code>src/utf/shadow_thirst/</code>	3	262	Mutation validation
TSCG Parser	<code>src/utf/tscg/</code>	3	139	Symbolic grammar
TSCG-B Codec	<code>src/utf/tscg_b/</code>	3	135	Binary serialization

UTF Examples — Proof of Concept Walkthroughs

The following examples demonstrate UTF capabilities across all tiers, from basic Thirsty-Lang syntax through Shadow Thirst mutation validation. All examples are executable via the UTF CLI.

Example: Basic Hello World

File: src/utf/examples/hello.thirsty

```
glass main() -> Int {  
  pour("hello, thirsty world");  
  return 0;  
}
```

Execute: python -m src.utf.thirsty_lang hello.thirsty

Example: Advanced: Classes, Async, Error Handling

File: src/utf/examples/gods.thirstofgods

```
fountain WaterTracker {  
  drink mut count: Int;  
  
  glass init(start: Int) {  
    this.count = start;  
  }  
  
  glass add(amount: Int) -> Int {  
    this.count = this.count + amount;  
    return this.count;  
  }  
  
  cascade glass current() -> Int {  
    return this.count;  
  }  
}  
  
glass main() -> Int {  
  drink mut tracker: WaterTracker = new WaterTracker(3);  
  pour(tracker.add(4));  
  pour(await tracker.current());  
  spillage {  
    throw "oops";  
  } cleanup error (e: Error) {  
    pour(e);  
  }  
  return 0;  
}
```

```
}
```

Execute: python -m src.utf.thirsty_lang gods.thirstofgods

Example: T.A.R.L. Policy

File: src/utf/examples/policy.tarl

```
policy PromotionGate {  
  when actor.role == "builder" and mutation.risk <= 3 => ALLOW;  
  when mutation.risk > 3 => ESCALATE;  
  when actor.role != "builder" => DENY;  
}
```

Execute: python -m src.utf.thirsty_lang policy.tarl

Example: Shadow Thirst Mutation

File: src/utf/examples/promote.shadowthirst

```
mutation validated_canonical set_counter(value: Int) {  
  shadow {  
    drink mut temp: Int = value;  
    temp = temp + 1;  
  }  
  invariant {  
    length("ok");  
  }  
  canonical {  
    canonical_counter = value;  
  }  
}
```

Execute: python -m src.utf.thirsty_lang promote.shadowthirst

Table of Contents

[Executive Overview](#1-executive-overview)

[Conceptual Foundation — The Water-Language Metaphor](#2-conceptual-foundation)

[UTF — Universal Thirsty Family](#3-utf-universal-thirsty-family)

[Thirsty-Lang](#4-thirsty-lang)

[T.A.R.L. / TARL](#5-tarl--tarl)

[TARL OS](#6-tarl-os)

[TSCG and TSCG-B](#7-tscg-and-tscg-b)

[Shadow Thirst](#8-shadow-thirst)

[Compiler / Parser / Runtime Pipeline](#9-compiler--parser--runtime-pipeline)

[Governance Integration](#10-governance-integration)

[Security Model](#11-security-model)

[Examples and Recipes](#12-examples-and-recipes)

[Developer Reference](#13-developer-reference)

[Current Implementation Status](#14-current-implementation-status)

[Roadmap](#15-roadmap)

[Glossary](#16-glossary)

[Source Map](#17-source-map)

[Appendix](#18-appendix)

1. Executive Overview

What is Thirsty-Lang?

Thirsty-Lang is a water-metaphor programming language that serves as the human-readable surface syntax for governance-bound execution inside Project-AI. It is not a toy DSL. It is the source-language tier of a multi-layer language family whose purpose is to make secure, auditable, policy-governed computation expressible in code that reads like intention rather than mechanism.

Thirsty-Lang programs are written in `.thirsty` files. The language has a lexer, a parser, an AST, a type checker, and a tree-walking interpreter — all implemented in Python inside the `src/utf/thirsty_lang/` module. A parallel, older JavaScript/Node.js implementation exists in `src/thirsty_lang/src/`. Both are live in the repository; the Python/UTF implementation is the newer and more architecturally integrated one.

What is UTF?

UTF — the **Universal Thirsty Family** — is the name for the complete umbrella of languages, encodings, runtimes, and symbolic systems that together carry Project-AI's execution model from source intent through policy enforcement to cryptographic audit. The family comprises six tiers, documented in `src/utf/docs/CANONICAL_STACK.md`:

Thirsty-Lang

Thirst of Gods (advanced dialect)

T.A.R.L.

Shadow Thirst

TSCG

TSCG-B

These are not independent tools. They are layers of a single execution stack. Source code written in Thirsty-Lang passes through governance checking in T.A.R.L., safe mutation simulation in Shadow Thirst, symbolic encoding in TSCG, binary transport in TSCG-B, and runtime enforcement in TARL OS — all governed by the Iron Path executor, the Sovereign Runtime, and Project-AI's constitutional enforcement machinery.

Why Does This Exist?

Project-AI is a governance-first AI framework. Its central thesis is: **governance precedes execution**. No action takes place unless policy explicitly allows it. No state mutation is committed unless it has passed Shadow simulation. No pipeline runs unless its artifacts are cryptographically signed and its audit trail is intact.

Thirsty-Lang and the UTF provide the language substrate for expressing, checking, encoding, and enforcing this guarantee across the entire stack — from a human writing a `.thirsty` script to a binary TSCG-B frame landing on a runtime.

2. Conceptual Foundation

The Water-Language Metaphor

Thirsty-Lang maps its computation model onto the metaphysics of water. This is not decoration. The metaphor encodes a design intent: code should carry **flow**, **containment**, **purity**, and **controlled release** as first-class concepts.

Water Concept	Computational Meaning
thirst	Conditional need — an <code>if</code> -like guard; the system "thirsts" for a condition
drink	Variable binding — data is "drunk into existence" from the environment
pour	Output / side-effect — data "pours" to the outside world
sip	Input — reading from the user or environment
glass	Function — a contained vessel that holds computation
fountain	Class — the blueprint for a living container
reservoir	Mutable data store — a named holding area for data under pressure
well	Immutable or read-only store — deep but stable
refill	Loop — the loop "refills" until exhausted
hydrated	Else branch — the condition is "hydrated" (the alternative path)
parched	Error or failure state — the system is "parched" and cannot continue
quenched	Satisfied / success state
flood	Bulk mutation — a <code>flood</code> rewrites many values at once
drip	Incremental update — a small controlled mutation
evaporate	Destructure or dissolve an optional — the value disappears safely
condense	Aggregate from parts — the inverse of evaporate
shield	Security container — everything inside is guarded
sanitize	Input validation — clean incoming data before use
armor	Output protection — mark state as immutable post-write
morph	Dynamic code transformation — obfuscate or adapt under attack
detect	Threat detection block — watch for attack patterns
defend	Reactive defense — apply a security posture in response

From `src/utf/docs/SACRED_TEXTS.md`:

*Variables are not merely declared. They are **drunk into existence**.*

*Data is not only stored. It is held in **reservoirs** and **wells**.*

*Bulk mutation is not a bland utility call. It is a **flood**.*

*Optional values do not disappear into null fog. They become **Quenched[T]** and may be **condensed** or **evaporated**.*

The Constitutional Framing

From `tarl_os/security/thirstys_constitution.thirsty`:

"Security as law, not heuristics."

"No action that affects another human may be non-replayable."

This is the governing axiom of the entire stack. Security is not a feature added to Thirsty-Lang. It is the reason the language exists.

3. UTF — Universal Thirsty Family

3.1 Definition

The Universal Thirsty Family (UTF) is the complete set of languages, formats, runtimes, and symbolic systems built on the water-metaphor paradigm. The canonical definition appears in `src/utf/docs/CANONICAL_STACK.md`. The UTF README (`src/utf/README.md`) describes it as "the amplified bootstrap implementation of the canonical family order."

3.2 Family Tree

```
UTF — Universal Thirsty Family
■
■■■ 1. Thirsty-Lang                (.thirsty)
■   Source language. Human-readable. Water-keyword syntax.
■   The entry point for all program expression.
■
■■■ 2. Thirst of Gods              (.thirstofgods, .thirstyplus, .thirstyplusplus)
■   Advanced dialect of Thirsty-Lang.
■   Adds: fountain classes, glass methods, cascade async,
■         spillage/cleanup error handling, new/this/await.
■
■■■ 3. T.A.R.L.                   (.tarl policy files)
■   Policy/runtime layer.
■   Evaluates deterministic ALLOW | DENY | ESCALATE rules
■   against context records. Governance enforcement tier.
■
■■■ 4. Shadow Thirst              (.shadowthirst)
■   Mutation simulation layer.
■   Enforces shadow → invariant → canonical separation.
■   Prevents unsafe state mutation from reaching canonical.
■
■■■ 5. TSCG                       (symbolic text expressions)
■   Thirsty's Symbolic Constitutional Grammar.
■   Compact, deterministic symbolic encoding of governance
■   flows: COG, DNT, SHD, INV, CAP, QRM, COM, ANC, RFX.
■
■■■ 6. TSCG-B                     (binary wire format)
■   Binary frame protocol for TSCG.
■   Wire format with CRC32 + SHA-256 integrity.
■   Magic: TSGB, Version 1, opcode-based payload.
```

3.3 Layer Roles

Member	Layer Type	File Extension	Role
Thirsty-Lang	Source language	.thirsty	Human-authored programs
Thirst of Gods	Source dialect	.thirstofgods, .thirstyplus, .thirstyplusplus	OOP + async programs
T.A.R.L.	Policy language / runtime	.tarl	Governance rule evaluation

Member	Layer Type	File Extension	Role
Shadow Thirst	Intermediate simulation	.shadowthirst	Safe mutation validation
TSCG	Symbolic grammar / encoding	text expression	Compact governance flow encoding
TSCG-B	Binary wire format	binary frames	On-wire/at-rest binary encoding

3.4 The Amplified Bootstrap

The `src/utf/` module is described as an "amplified bootstrap" — not a claim of full industrial completeness but a living, executable stack that contains the soul of the language in syntax, tooling, diagnostics, and flow. The following are confirmed executable as of the repository's last verified date (2026-04-20):

- Thirsty-Lang / Thirst of Gods: lexer, parser, AST, checker, interpreter, CLI, REPL, formatter, doctor, scaffold, bench, promote bridge, true tail-call optimization, namespace module imports.
 - T.A.R.L.: deterministic policy parser/evaluator.
 - Shadow Thirst: mutation parser, analyzer suite, replay/promote scaffolding, visualization, plugin loader.
 - TSCG: symbolic parser/encoder/decoder/checksum.
 - TSCG-B: binary frame codec with CRC32 + SHA-256.
-

4. Thirsty-Lang

4.1 Purpose

Thirsty-Lang is the human-readable source tier of the UTF. It exists to make governance-bound computation writable and readable. It is defensive by design: the language's security keywords (`shield`, `sanitize`, `armor`, `morph`, `detect`, `defend`) are first-class syntax, not libraries.

4.2 Design Philosophy

From `src/thirsty_lang/README.md`:

*"Thirsty-lang is a unique, expressive programming language designed to be **defensive and combative** against all known code threats."*

From `src/utf/docs/THIRST_MANIFESTO.md`:

"Code should feel like a living current, not dead syntax."

"Every command should feel poured, not bolted on."

"Metaphor should increase clarity, not obscure it."

4.3 Language Editions

Thirsty-Lang has four documented editions of increasing capability:

Edition	File Extension	Key Additions
Base (Thirsty-Lang)	<code>.thirsty</code>	Core keywords, basic control flow
Thirsty+	<code>.thirstyplus</code>	Enhanced control flow
Thirsty++	<code>.thirstyplusplus</code>	Functions with full type signatures
Thirst of Gods	<code>.thirstofgods</code>	Classes, <code>async</code> , OOP, error handling

4.4 Keywords

Core Keyword Table

Keyword	Meaning	Layer	Example	Notes
<code>drink</code>	Variable declaration (mutable)	Language	<code>drink x = 42</code>	Water metaphor for binding

Keyword	Meaning	Layer	Example	Notes
mut	Mutable modifier	Language	drink mut y: Int = 0	Explicit mutability
pour	Output / print	Language	pour "hello"	Side effect keyword
sip	Input from user/env	Language	sip username	Reads from stdin
thirsty	If / conditional	Language	thirsty x > 0 { ... }	Also used in .thirsty files as conditional
hydrated	Else branch	Language	hydrated { ... }	Antonym of parched
refill	Loop (for-style)	Language	refill (drink i = 0; i < 10; i += 1) { ... }	Loop construct
glass	Function definition	Language	glass greet(name) { ... }	Container for computation
fountain	Class definition	Language	fountain Person { ... }	Object blueprint
reservoir	Mutable data store	Language	reservoir store = {}	Named mutable container
well	Immutable store	Language	well config = { ... }	Read-only storage
flood	Bulk mutation	Language	flood store with updates	Mass assignment
drip	Incremental update	Language	drip x += 1	Small controlled mutation
evaporate	Dissolve optional	Language	evaporate value	Unwrap Quenched[T]
condense	Aggregate parts	Language	condense parts into result	Build from parts
parched	Failure / error state	Language	parched "error msg"	Error branch
quenched	Success state / option type	Type system	Quenched[T]	Wraps optional values
empty	Null optional	Type system	empty	Absence value
return	Function return	Language	return value	Exit glass block
import	Module import	Modules	import thirst::crypto	Namespace import
from	Import source	Modules	from pkg import sym	Selective import
as	Alias	Modules	import X as Y	Rename on import
shield	Security container	Security	shield componentName { ... }	Guarded execution zone
sanitize	Input validation	Security	sanitize userInput	Clean before use
armor	Immutability marker	Security	armor sensitiveData	Mark as immutable
morph	Dynamic transform	Security	morph on: ["injection"]	Code obfuscation
detect	Attack detection	Security	detect attacks { ... }	Threat watch block
defend	Defense posture	Security	defend with: "paranoid"	Apply defense strategy

Keyword	Meaning	Layer	Example	Notes
policy	Policy declaration	T.A.R.L.	policy Name { ... }	Governance rule block
when	Rule condition	T.A.R.L.	when expr => VERDICT;	Rule predicate
ALLOW	Permit verdict	T.A.R.L.	=> ALLOW	Access granted
DENY	Reject verdict	T.A.R.L.	=> DENY	Access blocked
ESCALATE	Escalate verdict	T.A.R.L.	=> ESCALATE	Require human review
mutation	Shadow mutation	Shadow Thirst	mutation validated _canonical ...	Safe mutation declaration
validated_canonical	Qualification keyword	Shadow Thirst	mutation validated _canonical X	Marks mutation as subject to validation
shadow	Shadow plane section	Shadow Thirst	shadow { ... }	Simulation-only block
invariant	Invariant check section	Shadow Thirst	invariant { ... }	Gate for promotion
canonical	Canonical commit section	Shadow Thirst	canonical { ... }	Real state write
promote	Approve mutation	Shadow Thirst	promote mutation	Commit after passing
reject	Block mutation	Shadow Thirst	reject mutation	Discard unsafe change
cascade	Async method modifier	Thirst of Gods	cascade glass fetch(url) { ... }	Async keyword
this	Self-reference	Thirst of Gods	this.name	OOP self
new	Object instantiation	Thirst of Gods	new Person("Alice")	Construct object
spillage	Error block	Thirst of Gods	spillage { ... } cleanup { ... }	try/catch equivalent
cleanup	Finally/catch block	Thirst of Gods	cleanup { ... }	Error cleanup
finally	Unconditional cleanup	Thirst of Gods	finally { ... }	Always-runs block
throw	Raise error	Thirst of Gods	throw Error("msg")	Error propagation
module	Module name declaration	Module header	module my_app	Optional; must precede all declarations
mode	Execution mode declaration	Module header	mode core / mode governed	Default is core
core	Core execution mode	Module header	mode core	Interpreter only; no governance gates
governed	Governed execution mode	Module header	mode governed	All TARL/Shadow/TSCG gates active
enum	Enum type declaration	Type system (Phase 3)	enum Status { Pending, Active }	Algebraic variant type
struct	Struct type declaration	Type system (Phase 3)	struct Point { x: Int, y: Int }	Named field record type

Keyword	Meaning	Layer	Example	Notes
interface	Interface declaration	Type system (Phase 3)	<code>interface Serializable { ... }</code>	Method contract for fountain classes
requires	Governance annotation	Governance (Phase 3)	<code>glass f() -> Governed[T] requires AuthorityClass.AC3</code>	Declares required authority/audit constraints

4.5 Operator Extensions

The `src/utf` implementation adds the following operator forms beyond the base spec:

Operator	Meaning	
<code>\</code>	<code>></code>	Pipeline pipe — pass left result as first arg to right
<code>thirst ... quench</code>	Guard expression – evaluate only if non-empty	
<code>sip?()</code>	Safe sip – returns <code>Quenched[T]</code> instead of throwing	
<code>pour?(...)</code>	Safe pour – fails silently if parched	

4.6 Type System

Source: `src/utf/docs/THIRSTY_TYPE_SYSTEM.md`, `src/utf/thirsty_lang/typesys.py`

Primitive Types

Type	Thirsty Name	Notes
Integer	<code>Int</code>	Python <code>int</code>
Float	<code>Float</code>	Python <code>float</code>
Boolean	<code>Bool</code>	Python <code>bool</code>
String	<code>String</code>	Python <code>str</code>
Void	<code>Void</code>	None – no return value
Any	<code>Any</code>	Unchecked; checker accepts any initializer
Error	<code>Error</code>	<code>ThirstyError</code> instance

Widening rules: `Int` widens to `Float`. All types widen to `Any`. No other implicit widening.

Core Generic Types (Phase 1 — Implemented)

`Quenched[T]` (Option/Maybe): Represents a value that may be absent. `empty` is the null value of any `Quenched[T]`. Unwrap with `condense(expr)` or the guard form `thirst expr quench`.

`Reservoir[T]` (Mutable Array): Ordered mutable collection. `drink mut xs: Reservoir[Int] = [1, 2, 3];`
 Functions: `size`, `push`, `pop`, `get`, `flood`.

`Task[T]` (Async Result): Returned by `cascade` glass functions. Unwrapped with `await`.

Extended Generic Types (Phase 3 — Implemented)

`Result[T, E]`: Represents success (T) or error (E). Functions that may throw declare `-> Result[T, E]`.
`spillage/cleanup` unwraps Result values.

`Governed[T]`: Return type for governance-annotated functions. Signals that the return value came from a governance-checked execution path. Only `requires`-annotated functions may return `Governed[T]`.

User-Defined Types (Phase 3 — Implemented)

```
// Enum
enum Status { Pending, Active, Closed }

// Struct
struct Account { id: Int, balance: Float, owner: String }

// Interface
interface Serializable { glass serialize() -> String }
```

The checker enforces: exhaustive enum matching, all struct fields required in constructors, fountain classes claiming an interface must implement all declared methods.

Governance Types (Phase 3 — Implemented)

`AuthorityClass`: Hierarchical authority levels AC1 (minimal) through AC5 (maximum). Declared in `requires` clauses on governed functions.

`AuditTrail.Immutable`: Marks that execution must produce an immutable audit record via the acceptance ledger.

`HumanAppealWindow[Nd]`: Marks that the decision produced by the annotated function is appealable for N days.

Type Checker Architecture

The type checker (`src/utf/thirsty_lang/checker.py`) implements lexical scoping, duplicate binding detection (E010), unknown identifier detection (E011), immutable assignment guards (E020), type mismatch checking (E021–E024), call arity validation (E030), pipe type compatibility, class field and method resolution, module import member access, and governance annotation validation (E050). Phase 3 adds enum exhaustiveness, struct field checking, interface implementation verification, and generic type variable substitution.

4.7 Execution Model

Thirsty-Lang programs are interpreted. The `src/utf/thirsty_lang/interpreter.py` module implements a tree-walking interpreter that walks the AST produced by the parser. Execution proceeds as:

Source Text

```

→ Lexer (lexer.py)      – produces Token stream
→ Parser (parser.py)    – produces AST
→ Checker (checker.py)  – type checking, scope resolution
→ Interpreter (interpreter.py) – tree-walk evaluation

```

The interpreter supports true tail-call optimization for user-function tail calls (confirmed in `src/utf/README.md`).

4.8 Security Keywords in Depth

Security keywords are not syntactic sugar. They map to real enforcement behaviors in the runtime and in TARL OS:

- `shield ComponentName { ... }` — Wraps an entire component or function body in a security enforcement zone. All operations inside are subject to threat detection and policy checking. Used pervasively in `.thirsty` files inside `tarl_os/`.
- `sanitize expr` — Validates and cleans an input value before it can be used. In TARL OS source files this is applied to every external parameter.
- `armor expr` — Marks state as immutable after writing. Prevents further mutation. Used on critical data structures like process tables, constitutional rules, and audit logs.
- `detect attacks { morph on: [...] defend with: "..."}` — Declares active threat detection. The `morph on:` clause lists attack vector names to watch for. The `defend with:` clause specifies the defense posture ("aggressive", "paranoid", "moderate", "careful").

4.9 Namespace Module Imports

Source: `src/utf/docs/THIRSTY_MODULE_SYSTEM.md`, `src/utf/thirsty_lang/module_system.py`

The import syntax requires a string literal, an `as` alias, and a terminating semicolon — all three are mandatory:

```

import "thirst::time" as t;
import "thirst::crypto" as crypto;
import "thirst::fs" as fs;
from "./path/file.thirsty" import name;

```

Note: The earlier form `import thirst::module` (no quotes, no alias, no semicolon) was the original sketch. The current, correct, implemented form uses string literals. All conformance fixtures and `stdlib` examples use the string-literal form.

Modules are cached on first import. The search path for relative imports starts at the directory of the importing file.

4.10 Module Header

Every `.thirsty` source file may begin with an optional module header that declares its name and execution mode:

```

module my_application
mode governed

```

Both lines are optional. If omitted, mode defaults to `core`. The header, if present, must appear before any declarations. Parsed into a `ModuleHeader` AST node stored on `Program.header`.

4.11 Execution Modes

Thirsty-Lang has two execution modes declared by the module header:

Mode	Parser	Checker	Interpreter	TARL	Shadow Thirst	TSCG
<code>core</code>	✓	✓	✓	—	—	—
<code>governed</code>	✓	✓	✓	✓	✓	✓

`mode core` programs run without the governance stack — useful for utility scripts, build tools, and test scaffolding. `mode governed` activates all TARL/Shadow Thirst/TSCG gates before execution and requires matching authority context to invoke governed functions.

At the CLI level: `thirsty run file.thirsty --authority AC3` injects the governance context.

4.12 Governance Annotations

Governance-annotated functions use `requires` clauses to declare their authority and audit requirements:

```
glass approve_task(task_id: String) -> Governed[String]
  requires AuthorityClass.AC3
  requires AuditTrail.Immutable
{
  drink sig: String = crypto.sign(task_id);
  return sig;
}
```

The checker validates `requires` clauses at parse time (THIRSTY-E050). The CLI's `thirsty govern` command reads these annotations and auto-generates a `.auto.tarl` policy file with matching TARL rules.

4.13 Stability Contract

Source: `STABILITY.md`

Thirsty-Lang v1.0.0 ships a formal stability contract:

- **Valid Core 1.x programs will not break.** Any program running under a 1.0 interpreter produces identical output under any 1.x interpreter.
- **Governed semantics may expand but not silently weaken.** A `DENY` verdict will never become `ALLOW` without an explicit policy change.
- **Security keywords cannot become no-ops.** `shield`, `sanitize`, `armor`, `morph`, `detect`, and `defend` will never be parsed and ignored.
- **Deny-by-default is semver-protected.** In `mode governed`, absence of a matching policy always produces `DENY`. Cannot be changed in a patch or minor release.

Versioning follows semver: PATCH = bug fixes only, MINOR = additive features (all 1.x programs remain valid), MAJOR = breaking changes with migration guide.

4.14 Error Code Registry

Source: `src/utf/docs/THIRSTY_ERROR_CODES.md`, `src/utf/thirsty_lang/diagnostics.py`

All toolchain errors carry a stable THIRSTY-Exxx code. Error objects include `code`, `message`, and `span` (with caret-marker formatted output). Multiple errors during type checking are reported as a `DiagnosticBundle` before exit.

Code	Category	Trigger
E001	Syntax	Unexpected token, invalid mode value, missing delimiter
E002	Lexer	Unterminated string literal, unrecognized character
E010	Scope	Duplicate drink binding in current scope
E011	Scope	Unknown identifier; offers nearest-match suggestion within edit distance 3
E020	Type	Assignment to immutable binding (missing mut)
E021	Type	Type mismatch at assignment, argument, return, pipe
E022	Type	Condition expression is not Bool
E023	Type	Loop count is not a non-negative Int
E024	Type	Return expression type incompatible with declared return type
E030	Call	Arity mismatch — wrong number of arguments
E031	Call	Call target is not callable
E032	Call	Subscript index is not Int
E050	Governance	Governed function called without sufficient AuthorityClass, or TARL returned DENY
E100	Runtime	Reservoir index out of bounds
E101	Runtime	Division by zero
E900	Limits	Recursion limit exceeded (256 frames; tail calls exempt via TCO)
E901	Limits	empty option unwrapped via condense or thirst/quench guard

5. T.A.R.L. / TARL

5.1 Name Disambiguation — A Documented Conflict

The repository contains two conflicting definitions of the T.A.R.L. acronym. This is documented here as a conflict, not silently resolved.

Source File	Definition
tarl/README.md, tarl/docs/ARCHITECTURE.md, tarl/docs/WHITEPAPER.md	Thirsty's Active Resistance Language
tarl_os/README.md (Integration section)	Trust and Authorization Runtime Layer

Both definitions appear in authoritative project documents. The `tarl/` module documentation (dated 2026-01-24, verified 2026-04-20) consistently uses "Thirsty's Active Resistance Language." The `tarl_os/README.md` (dated 2026-01-30, verified 2026-04-20) uses "Trust and Authorization Runtime Layer" in the integration section only.

Resolution for this document: The primary definition is **Thirsty's Active Resistance Language** per the `tarl/` module's own documentation. "Trust and Authorization Runtime Layer" appears to be a secondary or alternative framing used in integration contexts. Both are noted; neither is fabricated.

5.2 TARL as Policy Language (src/utf/tarl/)

The UTF stack contains a lightweight T.A.R.L. implementation in `src/utf/tarl/core.py`. This is the policy language tier of the UTF. It parses `.tarl` policy files and evaluates deterministic `ALLOW | DENY | ESCALATE` verdicts against context records.

Policy Syntax

```
policy PolicyName {
  when <boolean-expression> => ALLOW;
  when <boolean-expression> => ESCALATE;
  when <boolean-expression> => DENY;
}
```

Rules are evaluated in order. The first matching rule's verdict is returned. If no rule matches, the default verdict is **DENY** (fail-closed behavior).

Policy Evaluation

```
from tarl.core import parse_policy, evaluate

policy = parse_policy(source_text)
verdict = evaluate(policy, context_dict)
# verdict in {"ALLOW", "DENY", "ESCALATE"}
```

The expression evaluator (`SafeExpr`) uses a sandboxed Python AST evaluator that only permits a defined set of safe node types. Arbitrary function calls, attribute access outside dictionaries, and all non-safe AST nodes are rejected.

5.3 TARL as Full Language Runtime (tarl/)

The `tarl/` directory contains the production-grade language implementation — a full programming language with 8 subsystems.

What It Stands For Here

T.A.R.L. = **Thirsty's Active Resistance Language**. This is the primary, language-level meaning.

Architecture — 8 Subsystems

Layer 0: Configuration Registry	(tarl/config/)
Layer 1: Diagnostics Engine	(tarl/diagnostics/)
Layer 2: Standard Library	(tarl/stdlib/)
Layer 3: FFI Bridge	(tarl/ffi/)
Layer 4: Compiler Frontend	(tarl/compiler/)
Layer 5: Runtime VM	(tarl/runtime/)
Layer 6: Module System	(tarl/modules/)
Layer 7: Development Tooling	(tarl/tooling/)

Each layer depends only on layers below it. Zero circular dependencies.

Compiler Pipeline

Source Text	
→ Lexer	(tokenization, source location tracking)
→ Parser	(AST construction, error recovery)
→ Semantic	(type checking, scope resolution, symbol tables)
→ CodeGen	(bytecode emission, optimization passes)
→ Bytecode	(TARL_BYTECODE_V1 format)

Bytecode Format

Header: TARL_BYTECODE_V1\x00	(16 bytes)
Sections:	
- Constants pool	
- Code section	
- Debug info (optional, emitted when <code>debug_mode = true</code>)	

Bytecode is architecture-independent and deterministic.

Runtime VM

The VM is stack-based with:

- **Instruction dispatch:** interpreted loop with computed goto
- **Call stack:** function call frames and return addresses

- **Value stack:** operand stack for expression evaluation
- **Heap:** dynamic memory allocation (16MB default)
- **Garbage Collector:** mark-and-sweep with generational optimization (young/old generations, 75% heap threshold trigger)
- **JIT:** hot path compilation after 100 executions of a basic block

Configuration (tarl/config/tarl.toml)

```
[compiler]
debug_mode = false
optimization_level = 2
strict_mode = true

[runtime]
stack_size = 1048576    # 1MB
heap_size = 16777216    # 16MB
enable_jit = true
jit_threshold = 100

[security]
enable_sandbox = true
max_execution_time = 30.0
max_memory = 67108864    # 64MB
```

Security

The TARL runtime enforces sandboxing at the VM level:

- CPU time limit: 30 seconds (configurable)
- Memory: 64MB heap + 1MB stack (configurable)
- File I/O and network I/O: optional capability whitelists
- FFI: library allowlist, type validation, memory bounds checking
- Three FFI security modes: permissive, default, strict

Adapters

The tarl/adapters/ directory contains FFI adapters for:

- C# (tarl/adapters/csharp/TARL.cs)
- Go (tarl/adapters/go/tarl.go)
- Java (tarl/adapters/java/TARL.java)
- JavaScript (tarl/adapters/javascript/index.js)
- Rust (tarl/adapters/rust/lib.rs)

Integration with Cerberus and Codex Deus Maximus

From tarl/docs/ARCHITECTURE.md:

Cerberus detects a security threat.

A bridge (`src/app/agents/cerberus_codex_bridge.py`) analyzes the threat and maps it to T.A.R.L. features.

T.A.R.L. applies defensive compilation (basic, paranoid, or counter-strike mode).

Codex Deus Maximus implements permanent security upgrades.

Logs are written to `data/tarl_protection/implementations.jsonl`.

Thirsty-Lang Security Modules Referenced

From `tarl/README.md`:

- Threat detection: `src/thirsty_lang/src/security/threat-detector.js`
- Code morphing: `src/thirsty_lang/src/security/code-morpher.js`
- Defense compilation: `src/thirsty_lang/src/security/defense-compiler.js`

Performance Benchmarks (from whitepaper)

Metric	Value
Compilation speed	~50,000 lines/second
Interpreted execution	~1M instructions/second
JIT execution	~10M instructions/second
Cold startup	< 100ms
Warm startup (cached modules)	< 10ms
Base memory footprint	< 10MB

TarLRuntime (`tarl/runtime.py`)

The `TarLRuntime` class in `tarl/runtime.py` provides a high-performance policy evaluation runtime separate from the full T.A.R.L. system. It evaluates `TarLPolicy` objects against context dictionaries with:

- LRU-cached decisions (128 entries default)
- Parallel policy evaluation via thread pool
- Adaptive policy ordering (fastest policies first)
- Verdicts: `ALLOW`, `DENY`, `ESCALATE`

```
from tarl import TarLRuntime
from tarl.policy import TarLPolicy
from tarl.spec import TarLVerdict

runtime = TarLRuntime(policies=[...])
decision = runtime.evaluate(context)
# decision.verdict in TarLVerdict.{ALLOW, DENY, ESCALATE}
```


6. TARL OS

6.1 What is TARL OS?

TARL OS is described as a "God Tier AI Operating System" implemented in Thirsty-Lang / T.A.R.L. It is a monolithic AI operating substrate — all subsystems written in `.thirsty` files — providing the execution environment for AI workloads under full governance enforcement.

Source: `tarl_os/README.md` (version 2.0, status: Production Ready, last updated 2026-01-30). The implementation report (`tarl_os/TARL_OS_COMPLETE_IMPLEMENTATION_REPORT.md`) claims version 3.0 with 100% completeness, 29 production-grade subsystems, and approximately 13,600 lines of Thirsty-Lang code.

6.2 Architecture — 7 Tiers

```
TARL OS v3.0 – Complete Architecture

TIER 7: USER INTERFACE LAYER
  ■■■ Web Dashboard (dashboard.thirsty)
      Real-time monitoring, 6 widgets, 3 layouts,
      WebSocket updates, REST API, authentication

TIER 6: API LAYER
  ■■■ REST API (rest.thirsty)
  ■■■ gRPC API (grpc.thirsty)
  ■■■ GraphQL API (graphql.thirsty)
  ■■■ CLI (cli.thirsty)

TIER 5: OBSERVABILITY LAYER
  ■■■ Distributed Tracing (tracing.thirsty)    – OpenTelemetry compatible
  ■■■ Centralized Logging (logging.thirsty)    – 7 levels, structured
  ■■■ Alert Management (alerting.thirsty)      – rules, routing, escalation
  ■■■ Telemetry (telemetry.thirsty)           – metrics collection

TIER 4: AI ORCHESTRATION LAYER
  ■■■ AI Orchestrator (orchestrator.thirsty)   – workflow orchestration
  ■■■ Model Registry (model_registry.thirsty) – model version management
  ■■■ Feature Store (feature_store.thirsty)    – feature engineering
  ■■■ Inference Engine (inference.thirsty)    – inference serving

TIER 3: I/O LAYER
  ■■■ Virtual Filesystem (filesystem.thirsty)
  ■■■ Network Stack (network.thirsty)
  ■■■ Device Abstraction (devices.thirsty)

TIER 2: DEPLOYMENT LAYER
  ■■■ Deployment Orchestrator (deployment/orchestrator.thirsty)
  ■■■ Hot Update System (update.thirsty)
  ■■■ CI/CD Integration (cicd.thirsty)

TIER 1: KERNEL + SECURITY (Foundation)
  ■■■ Process Scheduler (kernel/scheduler.thirsty)
  ■■■ Memory Manager (kernel/memory.thirsty)
  ■■■ Secrets Vault (security/secrets_vault.thirsty)
```

```
■■■ RBAC System (security/rbac.thirsty)
■■■ Thirsty's Security Constitution (security/thirstys_constitution.thirsty)
■■■ Thirsty's Enforcement Gateway (security/thirstys_enforcement_gateway.thirsty)
■■■ Asymmetric Security (security/thirstys_asymmetric_security.thirsty)
■■■ Config Registry (config/registry.thirsty)
```

6.3 Kernel Subsystems

Process Scheduler (kernel/scheduler.thirsty)

Multi-level feedback queue scheduler:

- 8 priority levels (0 = REALTIME, 7 = IDLE)
- Preemptive multitasking with configurable quantum (default 100)
- CPU affinity support
- Priority aging to prevent starvation (boost when wait_time > 500)
- O(1) scheduling decisions
- Context switching with state preservation

Key functions: `initScheduler()`, `createProcess()`, `schedule()`, `terminateProcess()`, `getSchedulerStats()`

Memory Manager (kernel/memory.thirsty)

Paging-based memory management:

- 4KB page size
- Virtual memory support
- Page swapping and defragmentation
- Memory leak prevention
- O(1) allocation/deallocation

6.4 Security Subsystems

Thirsty's Security Constitution (security/thirstys_constitution.thirsty)

Five immutable constitutional rules enforced at runtime:

Rule ID	Name	Category	Action on Violation
rule_001	no_state_mutation_with_trust_decrease	state_integrity	HALT
rule_002	human_action_replayability	human_protection	HALT
rule_003	agent_audit_requirement	agent_accountability	HALT
rule_004	cross_tenant_authorization	multi_tenancy	HALT

Rule ID	Name	Category	Action on Violation
rule_005	privilege_escalation_approval	privilege_control	ESCALATE

The rules are locked with `armor constitutionalRules` during initialization and cannot be modified at runtime. A violation triggers either HALT (execution blocked, forensic snapshot created, escalation triggered) or ESCALATE (manual approval required).

From the constitution:

"Security is now law, not heuristics."

Thirsty's Enforcement Gateway (`security/thirstys_enforcement_gateway.thirsty`)

Truth-defining enforcement layer. Its guarantee:

`allowed=False` *means execution is IMPOSSIBLE.*

All operations pass through this gateway. If `enforcementActive` is false, the gateway fails closed with "Gateway offline - fail closed". Critical violations trigger forensic snapshots and escalation to the security team.

Secrets Vault (`security/secrets_vault.thirsty`)

- AES-256-GCM encryption
- Key rotation
- Automatic secret rotation
- Seal/unseal capability
- Access logging

RBAC (`security/rbac.thirsty`)

Five built-in roles: `super_admin`, `admin`, `operator`, `user`, `guest`. Role hierarchy with permission inheritance. Custom policy support. Comprehensive audit logging.

6.5 TARL OS Integration with T.A.R.L.

From `tarl_os/README.md`:

```
from tarl import TarlRuntime
from tarl_os.bridge import TARLOSBridge

tarl_runtime = TarlRuntime(DEFAULT_POLICIES)
tarl_os = TARLOSBridge()

def execute_with_tarl(operation, context):
    decision = tarl_runtime.evaluate(context)
    if decision.verdict == TarlVerdict.ALLOW:
        return tarl_os.execute_module_function(*operation)
    else:
        raise TarlEnforcementError(decision.reason)
```

TARL OS operations pass through TARL policy enforcement before execution. No TARL OS function executes without a verdict of ALLOW from the TarlRuntime.

6.6 Config Registry (`config/registry.thirsty`)

- 6 namespaces: system, security, AI, network, storage, user
 - Schema validation
 - Hot-reload support with watchers
 - Version tracking
 - Encryption for sensitive values
-

7. TSCG and TSCG-B

7.1 TSCG — Thirsty's Symbolic Constitutional Grammar

Source: `src/utf/tscg/core.py`, `src/utf/docs/TSCG_ROLE.md`

TSCG is the symbolic constitutional grammar used for compact deterministic state/execution expressions. It encodes governance flows — decisions, simulations, quorums, anchors, commits — as symbolic pipelines.

Core Symbols

Symbol	Mnemonic	Full Meaning
COG	Cognition	Cognition proposal
DNT	Delta non-terminal	Mutation proposal
SHD	Shadow	Deterministic shadow simulation
INV	Invariant	Invariant engine
CAP	Capability	Capability authorization
QRM	Quorum	Quorum
COM	Commit	Canonical commit
ANC	Anchor	Anchor (state binding)
REFX	Reflex	Reflex containment

Syntax

TSCG expressions use pipeline (->) and combination (^, ||) operators:

```
COG -> SHD -> INV -> COM
CAP ^ QRM -> COM
SHD || REFX -> ANC
DNT(arg1, arg2) -> INV -> COM
```

Parsing

The TSCG parser (`src/utf/tscg/core.py`) tokenizes and parses these expressions into an AST of `Symbol`, `Pipeline`, and `Combine` nodes. The canonical form of an expression is deterministic and used as the input to the SHA-256 checksum.

Checksum

```
from tscg.core import parse, checksum

expr = parse("COG -> SHD -> INV -> COM")
h = checksum(expr)
# h = sha256(canonical(expr))
```

Validation

Only recognized symbols are valid. Undefined symbols cause `ValueError: undefined TSCG symbol 'X'`. The recognized extended set includes: `ING`, `LED`, `SAFE`, `MUT`, `SEL`, `QRM_LINEAR`, `QRM_STATIC`.

Example: Governance Flow Encoding

A canonical governance decision that moves from cognition through shadow simulation through invariant checking to a canonical commit:

```
COG -> SHD -> INV -> COM
```

A quorum-gated capability authorization with anchor:

```
CAP ^ QRM -> ANC -> COM
```

A reflex containment with shadow fallback:

```
RFX || SHD -> INV -> COM
```

7.2 TSCG-B — Binary Frame Protocol

Source: `src/utf/tscg_b/core.py`, `src/utf/docs/TSCG_B_ROLE.md`

TSCG-B is the binary frame protocol for TSCG — the wire format for on-disk and on-network transport of TSCG expressions with integrity guarantees.

Frame Format

```
Bytes 0-3:  Magic = b"TSGB"
Byte  4:    Version = 0x01
Byte  5:    Flags = 0x00
Bytes 6-7:  Payload length (big-endian uint16)
Bytes 8..N: Encoded payload (opcode stream)
Bytes N..N+4: CRC32 of payload (big-endian uint32)
Bytes N+4..N+36: SHA-256 of canonical text (32 bytes)
```

Opcodes

Symbol	Opcode		
COG	0x02		
DNT	0x03		
SHD	0x04		
INV	0x05		
CAP	0x06		
QRM	0x07		
COM	0x08		
ANC	0x09		

Symbol	Opcode		
RFX	0x0B		
SAFE	0x0D		
-> (pipeline)	0x10		
^ (AND-combine)	0x11		
\	\	` (OR-combine)	0x14

Encoding / Decoding

```

from tscg_b.core import pack_text, unpack_frame

# Encode
frame = pack_text("COG -> SHD -> INV -> COM")

# Decode and verify
result = unpack_frame(frame)
# result = {
#   "version": 1,
#   "flags": 0,
#   "payloadLen": N,
#   "text": "COG -> SHD -> INV -> COM",
#   "crc32": "aabbccdd",
#   "sha256": "abcd...ef01"
# }
```

Decoding performs both CRC32 and SHA-256 verification. Any integrity failure raises `TSCGError`.

Integrity Guarantees

CRC32 validates payload byte integrity (protects against transmission errors).

SHA-256 validates the canonical text of the decoded expression (protects against semantic tampering).

Canonical normalization happens before hashing, so `COG->SHD` and `COG -> SHD` hash identically.

Streaming Decoder (Phase 5)

Source: `src/utf/tscg_b/core.py` — `StreamDecoder` class

The streaming decoder supports buffered multi-frame decoding without requiring a full frame to be present before beginning to process:

```

from tscg_b.core import StreamDecoder

decoder = StreamDecoder()
decoder.feed(chunk_of_bytes)           # feed partial data as it arrives
frames = decoder.frames()               # get all complete frames decoded so far
decoder.reset()                         # discard buffer and start fresh
# decoder.pending_bytes                 # number of bytes buffered but not yet decoded
```

The decoder performs magic-byte resynchronization — if the stream drifts or contains non-TSCG-B data, the decoder scans forward for the next `TSCG` magic byte and realigns. This makes the streaming decoder suitable for use in TCP streams and multiplexed channels where frame boundaries may not be guaranteed.

8. Shadow Thirst

8.1 What is Shadow Thirst?

Shadow Thirst is the mutation simulation and safety validation layer of the UTF. Its purpose is to prevent unsafe state changes from reaching the canonical (authoritative) execution plane. It does this by requiring every proposed mutation to pass through three ordered sections — `shadow`, `invariant`, `canonical` — before being committed.

From `src/utf/docs/SACRED_TEXTS.md`:

"Shadow Thirst is a constitutional simulation ritual: `shadow` → `invariant` → `canonical`."

8.2 Core Concept

The fundamental distinction in Shadow Thirst is between two planes:

Plane	Purpose	Can Write Canonical State?
Shadow plane	Simulation / speculation	NO — enforced by <code>PlaneIsolationAnalyzer</code>
Canonical plane	Authoritative state	YES — only after passing invariants

A mutation cannot write to canonical state from within the `shadow` block. This is a hard invariant enforced by static analysis at promote time.

8.3 Syntax

```
mutation validated_canonical MutationName(param: Type, ...) {
  shadow {
    // Safe simulation zone
    // May read canonical state but CANNOT write canonical_ variables
    // Must be deterministic (no now, rand, random)
    drink mut temp: Int = param;
    temp = temp + 1;
  }

  invariant {
    // Gate checks — must pass before canonical is touched
    length("ok");
  }

  canonical {
    // Actual state write — only reaches here if invariants pass
    canonical_counter = param;
  }
}
```

The keyword `validated_canonical` is required after mutation and signals that the mutation is subject to the full validation pipeline.

8.4 Analyzer Suite

Six built-in analyzers run at promote time:

Analyzer	Level	Checks
PlaneIsolationAnalyzer	critical	Shadow plane does not write canonical state
DeterminismAnalyzer	critical	Shadow plane has no non-determinism (no <code>now</code> , <code>rand</code> , <code>random</code>)
ResourceEstimator	warning	Estimated CPU $\leq 1000\text{ms}$, memory $\leq 256\text{MB}$
PuritySpringAnalyzer	critical	Invariant blocks remain pure
MemoryEvaporationAnalyzer	warning	Peak reservoir estimate $\leq 256\text{MB}$
CanonicalConvergenceAnalyzer	critical	Shadow and canonical must converge before promotion

Critical failures block promotion. Warning failures are noted but do not block.

8.5 Promote / Reject Flow

```
from shadow_thirst.core import parse_shadow, promote

module = parse_shadow(source_text)
result = promote(module, dry_run=False)

# result = {
#   "decision": "PROMOTE" | "REJECT",
#   "verdict": "PROMOTE" | "REJECT",
#   "dry_run": False,
#   "replay_id": "abcd1234...",
#   "analysis": [...],
#   "diff": "shadow spring aligns with the canonical river" | "the shadow river diverges...",
#   "replay_hash": "sha256hex..."
# }
```

A REJECT verdict means the mutation is withheld. It does not reach the canonical plane.

8.6 Replay Hash

Every promote decision produces a `replay_hash` — a SHA-256 digest of the mutation structure (names, params, body lengths). This makes every promotion decision reproducible and auditable.

8.7 Visualization

Shadow Thirst produces a Mermaid flowchart of the mutation pipeline:

```
flowchart TD
    set_counter_shadow["set_counter: shadow"] --> set_counter_inv["set_counter: invariant"]
    set_counter_inv --> set_counter_canon["set_counter: canonical"]
```

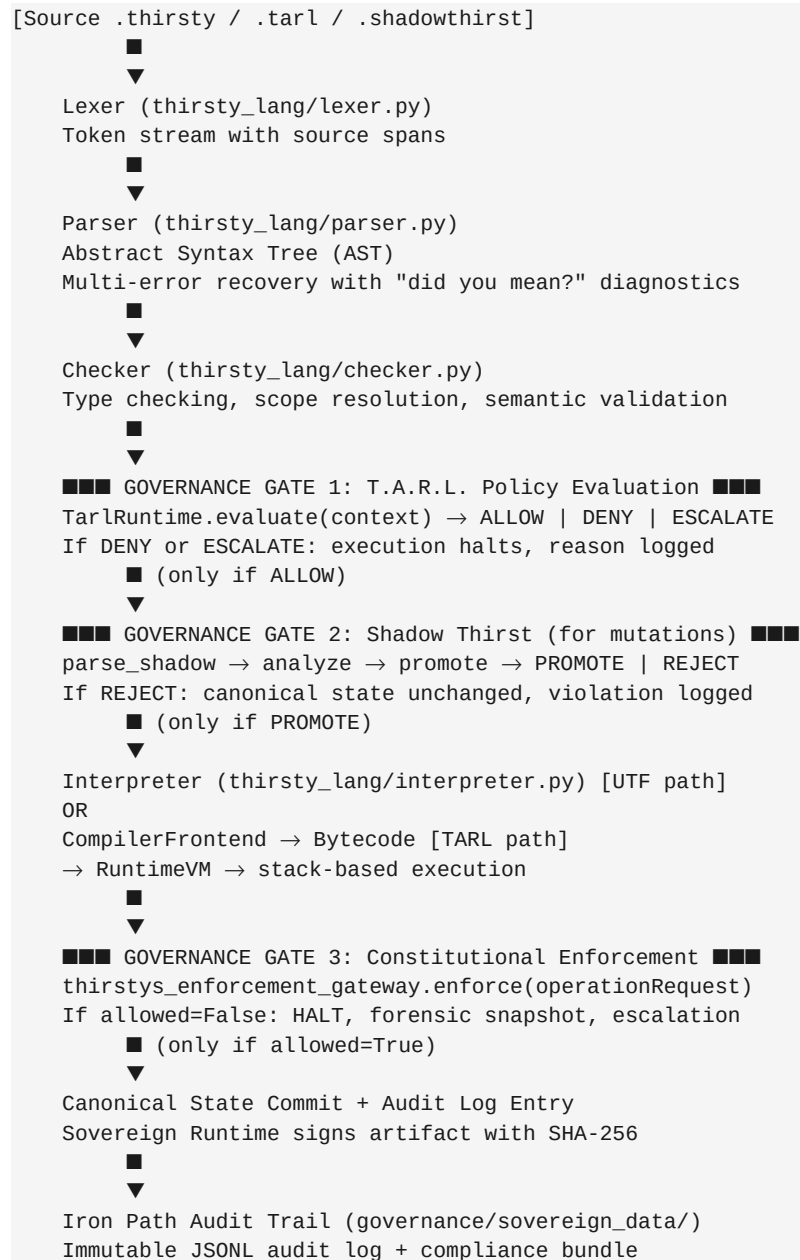
8.8 Plugin System

Shadow Thirst supports custom analyzers via a plugin loader that scans for `plugins/*.py` files that define an `analyze_plugin(module)` function. This allows project-specific invariant rules without modifying the core.

9. Compiler / Parser / Runtime Pipeline

9.1 End-to-End Architecture

The complete pipeline from source to audited state commit:



9.1b LLVM Backend

Source: `src/utf/tarl/compiler/frontend.py`, `src/utf/tarl/compiler/llvm_backend.py`

The Thirsty-Lang compiler now includes an LLVM backend via `llvmlite`. The `ThirstyFrontend` class lowers the AST to LLVM IR; the `LLVMBackend` class compiles IR to native artifacts.

Build target	CLI flag	Output
LLVM IR	<code>--target llvm-ir</code>	<code>dist/<module>.ll</code> – human-readable IR
Object file	<code>--target llvm-obj</code>	<code>dist/<module>.o</code> – native object file
Executable	<code>--target llvm-exe</code>	<code>dist/<module></code> – native binary
Assembly	<code>--target llvm-asm</code>	<code>dist/<module>.s</code> – native assembly
JIT execute	<code>--target llvm-jit</code>	Executes via MCJIT; no output file

The LLVM backend runs after the Thirsty type checker. It requires `llvmlite` (`pip install llvmlite`). All other build targets (`wasm-pyodide`, `js`) continue to operate without `llvmlite`.

9.1c Green Threads / Async

Source: `src/utf/thirsty_lang/interpreter.py`

`cascade glass` (async functions) now run concurrently. `TaskValue` was upgraded from a synchronous wrapper to a `concurrent.futures.Future`. All async functions execute in a shared `ThreadPoolExecutor`. `await` blocks on `future.result()` with a 30-second timeout. This is the first true concurrent execution model in Thirsty-Lang — prior to this, `cascade` functions ran synchronously.

9.2 Key Files by Pipeline Stage

Stage	File	Notes
Lexer	<code>src/utf/thirsty_lang/lexer.py</code>	Produces Token stream with Span tracking
Token definitions	<code>src/utf/thirsty_lang/token.py</code>	All keywords, operators, span type
Parser	<code>src/utf/thirsty_lang/parser.py</code>	AST construction
AST nodes	<code>src/utf/thirsty_lang/ast.py</code>	AST node definitions
Type checker	<code>src/utf/thirsty_lang/checker.py</code>	Semantic analysis
Interpreter	<code>src/utf/thirsty_lang/interpreter.py</code>	Tree-walking evaluation
TARL policy	<code>src/utf/tarl/core.py</code>	Policy parsing and evaluation
Shadow Thirst	<code>src/utf/shadow_thirst/core.py</code>	Mutation simulation and promotion
TSCG encoding	<code>src/utf/tscg/core.py</code>	Symbolic encoding
TSCG-B wire	<code>src/utf/tscg_b/core.py</code>	Binary transport
TARL compiler	<code>tarl/compiler/__init__.py</code>	Bytecode compilation
TARL runtime VM	<code>tarl/runtime/__init__.py</code>	Bytecode execution
Governance gate	<code>tarl/runtime.py</code>	<code>TarlRuntime</code> policy evaluation

Stage	File	Notes
Iron Path	<code>governance/iron_path.py</code>	Sovereign execution loop
Sovereign Runtime	<code>governance/sovereign_runtime.py</code>	Cryptographic signing, audit

9.3 Failure Behavior

Stage	Failure Mode	Result
Lexer	Invalid character	<code>ThirstyError THIRSTY-E001</code>
Lexer	Unterminated string	<code>ThirstyError THIRSTY-E002</code>
Parser	Syntax error	Structured diagnostic, multi-error recovery continues
Checker	Type mismatch	Structured diagnostic with suggestions
TARL Policy	DENY verdict	Execution blocked, reason logged
TARL Policy	ESCALATE	Blocked, requires human review
Shadow Thirst	Critical analyzer fail	REJECT — canonical state unchanged
Constitutional enforcement	Violation	HALT — forensic snapshot, escalate
Enforcement gateway offline	Any request	Fail closed — <code>allowed: false</code>
Bytecode verification	Integrity fail	<code>RuntimeError</code> , not executed
Audit trail	Integrity check fail	Execution summary flagged, logged

9.4 Validation Gates Summary

The Project-AI system enforces the principle **Default = DENY**:

- If T.A.R.L. does not explicitly return `ALLOW`, execution is denied.
- If Shadow Thirst does not return `PROMOTE`, the mutation is rejected.
- If the Constitutional Enforcement Gateway does not return `allowed: true`, the operation is blocked.
- If any audit trail entry fails integrity, the pipeline flags it.

Governance precedes execution at every layer.

10. Governance Integration

10.1 The Iron Path

The Iron Path (`governance/iron_path.py`) is the sovereign runtime demonstration and production execution mechanism. It runs a complete governance loop:

- Load pipeline (YAML) and create a config snapshot
- Verify config snapshot cryptographically
- Execute each stage with role signature + policy state binding
- Verify role signature and policy binding before each stage executes
- Generate SHA-256 artifact hash for each stage output
- Save stage artifact to disk
- Write to immutable audit log (JSONL)
- Produce compliance bundle on every successful run
- Verify audit trail integrity after completion

No stage executes without a verified role signature. No run completes without a compliance bundle. The Iron Path transforms Project-AI from "architecture" to "deployable sovereign system."

10.2 Sovereign Runtime

The `governance/sovereign_runtime.py` module provides:

- Cryptographic signing of role signatures
- Config snapshot creation and verification
- Policy state binding (hash-binding of policy state to execution context)
- Immutable audit log writes (append-only JSONL)
- Compliance bundle export
- Audit trail integrity verification

10.3 Triumvirate Governance

Source: `src/utf/docs/TRIUMVIRATE_SPEC.md`, `governance/triumvirate_server.py` — **Now fully implemented and documented.**

The Triumvirate is the runtime constitutional enforcement layer for all Legion agent intents. Before any high-stakes action (write, execute, mutate) is dispatched, the requesting actor submits an `IntentRequest` to the Triumvirate at port 8001. All three pillars vote independently. A single DENY from any pillar blocks the action permanently. Unanimous ALLOW is required for unrestricted execution.

The Three Pillars

Galahad — Ethics & Human Dignity: Checks intent text against 13 harm patterns (delete user, manipulate, deceive, override consent, surveil, blackmail, coerce, harm, destroy user, etc.). DENY on any match (confidence 0.95). ESCALATE on mutate with risk_level in ["high", "critical"]. ALLOW otherwise (confidence 0.90).

Cerberus — Security & Containment: Checks against 20 threat patterns (bypass, jailbreak, exfiltrate, inject, exploit, rootkit, backdoor, os.system, rm -rf, delete audit, etc.). DENY on match (confidence 0.98). ESCALATE on untrusted actor requesting execute/mutate, or external origin requesting write.

CodexDeus — Constitutional Law & FourLaws: Checks against 12 constitutional violation patterns (violate fourlaws, disable governance, self-modify constitution, dissolve triumvirate, etc.). DENY on match (confidence 0.99). ESCALATE on any mutate action (all mutations require constitutional review).

The FourLaws

Hard-coded constitutional constants — immutable, supersede all other rules:

- Legion must not harm humans or allow harm through inaction.
- Legion must obey human instructions unless they conflict with the First Law.
- Legion must protect Project-AI's constitutional integrity.
- Legion must act with transparency and honesty in all communications.

Decision Engine

Aggregation rules (strict priority order): any DENY → final verdict deny; any ESCALATE (no DENY) → escalate; all ALLOW → allow. The audit_id is a 16-char hex SHA-256 fingerprint of actor + action + target + timestamp.

API

Method	Path	Description
POST	/intent	Submit IntentRequest → receive GovernanceDecision
GET	/audit?limit=N	Most recent N decisions (in-memory ring buffer, max 1000)
GET	/fourlaws	Return the immutable FourLaws
POST	/chimera/verdict	Receive threat verdict from Chimera deception perimeter
POST	/chimera/canary	Receive canary-hit alert from Chimera perimeter

Governance Layer Relationship

COMPILE TIME	RUNTIME (static)	RUNTIME (dynamic)
checker.py requires clauses → THIRSTY-E050	interpreter._enforce_ governance() → ThirstyGovernanceError	Triumvirate /intent POST All three pillars vote → GovernanceDecision

The Thirsty interpreter enforces structural governance (annotation consistency). The Triumvirate enforces constitutional governance (ethics, security, FourLaws). Both layers must pass independently.

10.3b PSIA — Plane Separation / Isolation Architecture

Source: `src/utf/docs/PSIA_SPEC.md`, `src/psia/` — **Now fully implemented and documented. Acronym confirmed: Plane Separation / Isolation Architecture.**

PSIA is a 7-stage, 6-plane defense pipeline that provides monotonically increasing strictness from raw untrusted input to cryptographically sealed canonical state. No stage can be skipped, bypassed via feature flag, or softened in debug mode. Port 8002 (Triumvirate is 8001).

The 6 Planes

Plane	Name	Description
0	Entry	Raw untrusted input from the wire
1	Validated	Schema-checked, type-safe, structurally sound
2	Classified	Intent class and risk level assigned
3	Shadow	Parallel simulation; 4 invariant checks
4	Governed	Triumvirate constitutional evaluation completed
5	Canonical	Written to append-only hash-chained log
6	Sealed	Merkle-anchored, Ed25519-signed

Each plane has an immutable dataclass (`RawFrame`, `ValidatedFrame`, `ClassifiedFrame`, `ShadowedFrame`, `GovernedFrame`, `CanonicalFrame`, `SealedFrame`) in `src/psia/schemas/models.py`. Stages produce new frame objects — they never mutate existing frames.

The 7 Stages

Stage 0 — Ingestion: Required fields: actor, action, target. Produces SHA-256 fingerprint.

Stage 1 — Schema Validation: Valid actors: human, agent, system. Valid actions: read, write, execute, mutate.

Stage 2 — Classification: Heuristic risk ladder — `critical/rm -rf` → critical; `destroy/mutate` → high; `execute/write` → medium; `read` → low. Assigns `threat_score` (0.0–1.0).

Stage 3 — Shadow Simulation: 4 invariant checks: `PlaneIsolation` (no canonical write bypass), `Determinism` (identical inputs → identical shadow hash, verified by running simulation twice), `ResourceBound` (cost ≤ 1000 units), `Purity` (action is read-only or `threat_score` < 0.3). Rejection on any failure.

Stage 4 — Governance: Submits to Triumvirate at `http://localhost:8001/intent`. Any DENY → stage fails. Fallback if Triumvirate is unreachable: identical inline rule-based evaluation — no silent pass-through.

Stage 5 — Canonical Log: Append-only hash-chained JSONL log with monotonic sequence numbers. Chain integrity verifiable at any time.

Stage 6 — Seal: Proper Merkle tree over all canonical log entry hashes (SHA-256 leaves, paired internal nodes, inclusion proofs) + Ed25519 signature over `block_hash`. Ed25519 key loaded from env var or falls back to software-only mode (signature field is empty string — visible in output, not silently omitted).

Pre-Screen Gate

Before Stage 0, a fast $O(1)$ `PreScreenGate` rejects absolute prohibitions immediately (e.g., `disable triumvirate`, `jailbreak`, `delete audit`, `format drive`) without entering the full pipeline.

Pipeline API

```
from psia.core import Pipeline

pipeline = Pipeline(canonical_log_path=Path("canonical.jsonl"))
result = pipeline.run({"actor": "agent", "action": "execute", "target": "..."})

if result.passed:
    print(f"Sealed: {result.sealed_hash}")
    print(f"Merkle: {result.sealed.merkle_root}")
else:
    print(f"Denied at stage {result.trace}: {result.error}")
```

PSIA vs Shadow Thirst

Shadow Thirst (`src/utf/shadow_thirst/`) simulates `.shadowthirst` mutation blocks written in Thirsty-Lang. PSIA Stage 3 simulates intent requests flowing through the HTTP gateway. Both share the plane-separation philosophy — shadow never writes canonical — but are independent systems with separate codebases.

10.4 Cerberus

Cerberus is the threat detection agent. Referenced in `tarl/docs/ARCHITECTURE.md` and `tarl/docs/WHITEPAPER.md`. The integration bridge is `src/app/agents/cerberus_codex_bridge.py`. Cerberus detects threats → T.A.R.L. applies defensive compilation → Codex implements permanent upgrades.

Cerberus source is in `src/cerberus/` (directory confirmed present).

10.5 Codex Deus Maximus

Codex Deus Maximus orchestrates permanent security feature integration. Referenced in `tarl/docs/ARCHITECTURE.md`. Logs to `data/tarl_protection/implementations.jsonl`. Workflow files: `.github/workflows/codex-deus-ultimate.yml`. Full implementation details are in `src/` subdirectories.

10.6 PSIA Planes

`src/psia/` is present in the repository. PSIA (Plane Separation / Isolation Architecture — exact acronym expansion: **undocumented / needs confirmation**) is referenced. The plane separation model (shadow vs. canonical) is fully implemented in Shadow Thirst. Whether PSIA has an independent module beyond this is undocumented.

10.7 Governance Audit Trail

Every governance operation writes to `governance/sovereign_data/immutable_audit.jsonl`. Artifacts are written to timestamped subdirectories under `governance/sovereign_data/artifacts/`. The Sovereign Runtime verifies audit trail integrity by checking each entry's hash chain after execution.

11. Security Model

11.1 Threat Model

The Project-AI security model assumes:

- Adversarial inputs at every API surface
- Injection attacks (SQL, command, XSS, etc.)
- Privilege escalation attempts
- Cross-tenant data access
- State mutation without authorization
- Non-replayable actions affecting humans
- Agent actions without audit spans

11.2 Defense Layers

Layer 1: Language-Level (Thirsty-Lang)

Security keywords are syntax, not libraries:

- `sanitize` on every external input
- `armor` on every sensitive output
- `shield` around every component
- `detect` attacks with explicit attack vector enumeration
- `defend` with: "paranoid" posture on critical paths

Layer 2: Policy-Level (T.A.R.L.)

All context-sensitive access decisions are evaluated by T.A.R.L.:

- Default = DENY (no matching rule = blocked)
- Rules are deterministic and sandboxed
- SafeExpr evaluator rejects any non-safe AST node
- Policy files are version-controlled and audited

Layer 3: Mutation-Level (Shadow Thirst)

All proposed state changes pass through Shadow simulation:

- `PlaneIsolationAnalyzer`: shadow cannot write canonical state

- DeterminismAnalyzer: shadow must be deterministic
- CanonicalConvergenceAnalyzer: must converge before promotion
- Rejected mutations never reach canonical state

Layer 4: Runtime-Level (TARL VM + Sandbox)

- CPU time limit: 30 seconds
- Memory limit: 64MB heap
- Stack overflow protection
- Heap bounds checking
- Safe garbage collection
- No arbitrary code execution from within VM

Layer 5: Constitutional-Level (Thirsty's Constitution)

5 immutable constitutional rules enforced at every operation:

- No simultaneous state mutation + trust decrease
- All human-affecting actions must be replayable
- All agent actions must have an audit span
- Cross-tenant actions require explicit authorization
- Privilege escalation requires ≥ 2 approvers

Layer 6: Gateway-Level (Enforcement Gateway)

Truth-defining enforcement: `allowed=False` means physically impossible execution. Gateway offline = fail closed.

Layer 7: Cryptographic-Level (Sovereign Runtime)

Every stage artifact is SHA-256 hashed. Every role signature is cryptographically verified. Every config snapshot is verified before execution. The audit trail is an append-only JSONL with integrity verification.

11.3 What Happens When Validation Fails

Failure	System Response
TARL DENY	Log reason, block execution, return TarlDecision(DENY)
TARL ESCALATE	Block, log, route to human reviewer
Shadow REJECT	Canonical unchanged, replay_hash logged, diff includes rejection reason
Constitutional HALT	Execution blocked, forensic snapshot created, violation recorded, escalation triggered
Gateway offline	Fail closed — all requests blocked
CRC32 mismatch in TSCG-B	TSCGBError raised, frame rejected

Failure	System Response
SHA-256 mismatch in TSCG-B	TSCGBError raised, frame rejected
Role signature verification fail	RuntimeError, stage does not execute
Policy binding verification fail	RuntimeError, stage does not execute

12. Examples and Recipes

12.1 Basic Thirsty-Lang Program

```
// hello.thirsty
glass main() -> Int {
  pour("hello, thirsty world");
  return 0;
}
```

Source: src/utf/examples/hello.thirsty

Expected behavior: Prints hello, thirsty world and returns 0.

Failure mode: If pour is blocked by enforcement gateway (e.g., not in allowed output context), the operation returns allowed: false.

12.2 Governed Agent Runner (Phase 3–4 Showcase)

A complete mode governed program demonstrating Governed[T] return types, requires annotations, AuthorityClass, AuditTrail.Immutable, and HumanAppealWindow. This is the canonical showcase for the governance execution model.

```
// src/utf/examples/showcase/governed_agent_runner/main.thirsty
module governed_agent_runner
mode governed

import "thirst::crypto" as crypto;
import "thirst::time" as t;
import "thirst::env" as env;

glass log_step(msg: String) -> Void {
  pour(msg);
}

glass approve_task(task_id: String) -> Governed[String]
  requires AuthorityClass.AC3
  requires AuditTrail.Immutable
{
  drink ts: Int = t.now();
  drink sig: String = crypto.sign(task_id);
  pour("APPROVED");
  return sig;
}

glass review_task(task_id: String) -> Governed[String]
  requires AuthorityClass.AC2
  requires HumanAppealWindow[30d]
{
  drink h: String = crypto.sha256(task_id);
  pour("REVIEWED");
  return h;
}
```

```

glass run_agent(task_id: String) -> Int {
  log_step("--- Governed Agent Runner ---");
  log_step("Step 1: Identify task");
  pour(task_id);

  log_step("Step 2: Review phase (AC2 required)");
  drink review_result: Any = review_task(task_id);
  pour(review_result);

  log_step("Step 3: Approval phase (AC3 required)");
  drink approval: Any = approve_task(task_id);
  pour(approval);

  log_step("Step 4: Execution complete");
  pour("DONE");
  return 0;
}

glass main() -> Int {
  drink task: String = "agent-task-001";
  return run_agent(task);
}

```

Source: src/utf/examples/showcase/governed_agent_runner/main.thirsty

Run with: thirsty run --demo governed_agent --authority AC3

What it demonstrates:

- module/mode header — mode governed activates all governance gates
- Governed[String] return type — signals governance-checked execution path
- requires AuthorityClass.AC3 — caller must hold AC3 authority or higher
- requires AuditTrail.Immutable — every invocation writes an immutable audit record
- requires HumanAppealWindow[30d] — decisions are appealable for 30 days
- Multi-namespace imports from stdlib (crypto, time, env)
- Auto-generates governed_agent_runner.auto.tarl policy via thirsty govern

12.2 Variable Declaration and Output

```

drink greeting = "Hello, World!"
drink count = 42
pour greeting
pour count

```

Expected behavior: Outputs the greeting string then the integer.

12.3 Conditional Flow

```

drink age = 25

thirsty age > 18 {
  pour "Adult"
}

```



```

} hydrated {
  pour "Minor"
}

```

12.4 Loop

```

refill (drink i = 0; i <= 7; i += 1) {
  pour "Level: " + i
}

```

Source pattern from: `tarl_os/kernel/scheduler.thirsty` (queue scan loop).

12.5 Governance-Gated Policy Declaration (T.A.R.L.)

```

policy PromotionGate {
  when actor.role == "builder" and mutation.risk <= 3 => ALLOW;
  when mutation.risk > 3 => ESCALATE;
  when actor.role != "builder" => DENY;
}

```

Source: `src/utf/examples/policy.tarl`

Expected behavior:

- A builder with low-risk mutation: ALLOW
- Any actor with high-risk mutation: ESCALATE (blocked, human review)
- Non-builder actor: DENY (blocked)

Evaluation:

```

context = {"actor": {"role": "builder"}, "mutation": {"risk": 2}}
evaluate(policy, context) # → "ALLOW"

context = {"actor": {"role": "admin"}, "mutation": {"risk": 1}}
evaluate(policy, context) # → "DENY"

```

12.6 Shadow Mutation (Safe State Change)

```

mutation validated_canonical set_counter(value: Int) {
  shadow {
    drink mut temp: Int = value;
    temp = temp + 1;
    // Cannot write canonical_counter here — PlaneIsolationAnalyzer blocks
  }
  invariant {
    length("ok");
    // Gate: must evaluate cleanly before canonical is touched
  }
  canonical {
    canonical_counter = value;
    // This is the only place that may write canonical state
  }
}

```

```
}
```

Source: src/utf/examples/promote.shadowthirst

Expected behavior: promote() returns decision: "PROMOTE" — shadow is deterministic, does not write canonical state, invariant passes, canonical section assigns correctly.

Failure mode: If shadow block contained canonical_counter = value, PlaneIsolationAnalyzer would fail at critical level, and promote() would return decision: "REJECT".

12.7 Security-Guarded Component (TARL OS Pattern)

```
shield scheduler {
  drink processTable = {}
  armor processTable

  glass createProcess(command, priority, memory_required) {
    detect attacks {
      morph on: ["injection", "overflow", "privilege_escalation"]
      defend with: "aggressive"
    }
    sanitize command
    sanitize priority
    sanitize memory_required

    thirsty (priority < 0 or priority > 7) {
      pour "ERROR: Invalid priority level"
      return null
    }

    drink pcb = {
      pid: nextPID,
      command: command,
      priority: priority,
      status: "ready"
    }
    armor pcb

    processTable[nextPID] = pcb
    return pcb
  }
}
```

Source: Derived from tarl_os/kernel/scheduler.thirsty

Expected behavior: All inputs are sanitized. Priority bounds are checked. The PCB is armored on creation. Any injection or overflow attempt is morphed/defended against.

12.8 TSCG Symbolic Encoding

```
from tscg.core import parse, canonical, checksum

# Encode a governance flow
expr = parse("COG -> SHD -> INV -> COM")
print(canonical(expr)) # "COG -> SHD -> INV -> COM"
```

```
print(checksum(expr))    # sha256 hex

# Quorum-gated capability with anchor
expr2 = parse("CAP ^ QRM -> ANC -> COM")
print(checksum(expr2))
```

12.9 TSCG-B Binary Frame Round-Trip

```
from tscg_b.core import pack_text, unpack_frame

# Pack a governance flow into binary
frame = pack_text("COG -> SHD -> INV -> COM")
# frame = b"TSGB" + version + flags + length + payload + crc32 + sha256

# Unpack and verify
result = unpack_frame(frame)
print(result["text"])    # "COG -> SHD -> INV -> COM"
print(result["crc32"])   # 8-char hex
print(result["sha256"])  # 64-char hex
```

Failure mode: If any byte of the payload is corrupted, `unpack_frame` raises `TSCGBError: CRC mismatch`. If the canonical text SHA-256 does not match, raises `TSCGBError: SHA mismatch`.

12.10 Full End-to-End: Iron Path Execution

```
from governance.iron_path import IronPathExecutor

executor = IronPathExecutor(
    pipeline_path="sovereign-demo.yaml",
    artifacts_dir=Path("output/artifacts")
)
result = executor.execute()

# result = {
#   "execution_id": "uuid",
#   "status": "completed",
#   "stages_completed": [...],
#   "artifacts": {"data_preparation": "path/to/artifact.json", ...},
#   "hashes": {"data_preparation": "sha256hex", ...},
#   "audit_integrity": {"is_valid": True, "issues": []},
#   "compliance_bundle": {...}
# }
```

Each stage is executed only after:

- Role signature is verified
- Policy state binding is verified
- Stage output is SHA-256 hashed
- Artifact is written to disk
- Audit log entry is appended

13. Developer Reference

13.1 Repository Structure (Thirsty/UTF-Relevant)

```

Project-AI/
■■■■ build.tarl                # Thirsty-Lang build script
■■■■ tarl/                    # T.A.R.L. language runtime
■   ■■■■ README.md           # Main TARL documentation
■   ■■■■ docs/ARCHITECTURE.md # Full architecture
■   ■■■■ docs/WHITEPAPER.md  # Technical whitepaper
■   ■■■■ config/tarl.toml     # Configuration
■   ■■■■ core.py              # TARLSystem root
■   ■■■■ runtime.py           # TarlRuntime (policy evaluation)
■   ■■■■ policy.py            # TarlPolicy
■   ■■■■ spec.py              # TarlVerdict, TarlDecision
■   ■■■■ parser.py            # TARL parser
■   ■■■■ compiler/           # Bytecode compiler
■   ■■■■ runtime/            # VM runtime
■   ■■■■ adapters/{csharp,go,java,js,rust}/
■   ■■■■ tests/test_tarl_integration.py
■■■■ tarl_os/                  # TARL OS (AI Operating System)
■   ■■■■ README.md
■   ■■■■ TARL_OS_COMPLETE_IMPLEMENTATION_REPORT.md
■   ■■■■ kernel/scheduler.thirsty
■   ■■■■ kernel/memory.thirsty
■   ■■■■ security/thirstys_constitution.thirsty
■   ■■■■ security/thirstys_enforcement_gateway.thirsty
■   ■■■■ security/thirstys_asymmetric_security.thirsty
■   ■■■■ security/rbac.thirsty
■   ■■■■ security/secrets_vault.thirsty
■   ■■■■ config/registry.thirsty
■   ■■■■ ai_orchestration/{orchestrator,model_registry,feature_store,inference}.thirsty
■   ■■■■ api/{rest,grpc,graphql}.thirsty
■   ■■■■ observability/{telemetry,tracing,logging,alerting}.thirsty
■   ■■■■ io_layer/{filesystem,network,devices}.thirsty
■   ■■■■ deployment/{orchestrator,update,cicd}.thirsty
■   ■■■■ ui/dashboard.thirsty
■   ■■■■ tools/cli.thirsty
■   ■■■■ bridge.py            # Python execution bridge
■■■■ governance/
■   ■■■■ core.py              # GovernanceCore
■   ■■■■ iron_path.py         # IronPathExecutor
■   ■■■■ sovereign_runtime.py # Cryptographic runtime
■   ■■■■ sovereign_verifier.py # Artifact verification
■   ■■■■ triumvirate_server.py # Triumvirate governance (partial)
■■■■ src/
■■■■ utf/                    # Universal Thirsty Family bootstrap
■   ■■■■ README.md
■   ■■■■ docs/{CANONICAL_STACK,THIRSTY_LANG_SPEC,SHADOW_THIRST_SPEC,
■   ■■■■          TSCG_ROLE,TSCG_B_ROLE,SACRED_TEXTS,GREAT_WELLS,
■   ■■■■          THIRST_MANIFESTO,THIRST_OF_GODS_SPEC,TARL_BOUNDARY}.md
■   ■■■■ thirsty_lang/{lexer,parser,ast,checker,interpreter,
■   ■■■■          token,typesys,module_system,package_manager,
■   ■■■■          diagnostics,cli}.py
■   ■■■■ tarl/{core,cli}.py
■   ■■■■ shadow_thirst/{core,cli}.py

```

```
■ ■■■ tscg/{core,cli}.py
■ ■■■ tscg_b/{core,cli}.py
■ ■■■ tests/{test_thirsty_lang, test_tarl, test_shadow_thirst,
■           test_tscg, test_thirsty_flavor, test_package_gallery,
■           test_properties}.py
■ ■■■ examples/{hello.thirsty, policy.tarl, promote.shadowthirst,
■               context.json, flavor.thirsty, namespace_imports.thirsty,
■               gods.thirstofgods}
■■■ thirsty_lang/                # Original JS/Node.js implementation
■ ■■■ README.md
■ ■■■ src/{ast,cli,debugger,formatter,index,linter}.js
■ ■■■ docs/{SPECIFICATION,SECURITY_GUIDE,TUTORIAL,FAQ}.md
■ ■■■ examples/
■■■ cerberus/                    # Cerberus threat detection
■■■ psia/                        # PSIA plane isolation (TBC)
```

13.2 Standard Library

Source: src/utf/docs/THIRSTY_STANDARD_LIBRARY.md, src/utf/thirsty_lang/module_system.py

Phase 2 expanded the stdlib from 3 to **15 namespaces** plus 16 global built-ins. Import syntax (all three elements required):

```
import "thirst::module" as alias;
```

Global Built-ins (always available, no import)

length contains split abs min max push pop size get flood condense evaporate strain transmute distill

Stdlib Namespaces

Namespace	Import As	Key Functions
thirst::time	t	now(), epoch_ms()
thirst::crypto	crypto	sha256, sign, hmac, random_bytes, uuid4
thirst::reservoir	res	size, push, pop, get, flood
thirst::fs	fs	read_file, write_file, exists, list_dir, mkdir, remove
thirst::path	p	join, dirname, basename, extension, absolute, relative
thirst::json	json	parse, stringify, get, set
thirst::http	http	get, post, put, delete
thirst::env	env	get, set, all
thirst::process	proc	run, exit, args, pid
thirst::log	log	info, warn, error, debug
thirst::test	t	assert_eq, assert_ne, assert_true, assert_raises, describe, it
thirst::collections	col	map, filter, reduce, sort, unique, flatten, zip

Namespace	Import As	Key Functions
thirst::net	net	tcp_connect, tcp_listen, udp_send
thirst::sqlite	db	connect, query, execute, close
thirst::yaml	yaml	parse, dump
thirst::toml	toml	parse, dump

thirst::crypto.sign *replaces the earlier* crypto.bless *name*. All conformance fixtures use sign.

strain/transmute/distill (*functional equivalents of filter/map/reduce*) are implemented as global built-ins; the thirst::reservoir *namespace mirrors them by name but delegates to the global forms*.

13.3 Commands

```
# Run Thirsty-Lang program
thirsty run examples/hello.thirsty --trace --thirst-level 2

# Run with governance context
thirsty run --demo governed_agent --authority AC3

# Start REPL
thirsty repl

# Doctor check
thirsty doctor .

# Scaffold new app
thirsty new app my_project

# Format code (AST-based pretty-printer)
thirsty fmt file.thirsty

# Package management
thirsty add some_package
thirsty audit                # check dependency hashes
thirsty lock                  # generate thirsty.lock with SHA-256 per entry

# Build
thirsty build file.thirsty           # standard build
thirsty build file.thirsty --target js      # emit dist/<module>.js
thirsty build file.thirsty --target wasm-pyodide # emit self-contained browser HTML
thirsty build file.thirsty --emit-manifest   # full governance manifest with TSCG-B frame

# Governance
thirsty govern file.thirsty          # report annotation completeness + emit .auto.tarl

# LSP server
thirsty lsp                          # stdio mode (for editors)
thirsty lsp --port 9898              # TCP debug mode

# Docs site
thirsty docs                         # generate static HTML to dist/docs/
thirsty docs --output ./site         # custom output dir
thirsty docs --open                  # generate and open in browser

# LLVM compilation
```

```

thirsty build file.thirsty --target llvm-ir    # emit LLVM IR
thirsty build file.thirsty --target llvm-exe  # native binary
thirsty build file.thirsty --target llvm-jit  # JIT execute

# Package registry
# Set THIRSTY_REGISTRY_URL to use a custom registry endpoint
# Default: http://localhost:9000
# Start local registry:
py src/utf/thirsty_lang/registry_server.py
# Or: uvicorn src.utf.thirsty_lang.registry_server:app --port 9000

# Triumvirate governance server
python governance/triumvirate_server.py      # port 8001
# Or: uvicorn governance.triumvirate_server:app --port 8001

# PSIA pipeline gateway
uvicorn psia.server.app:app --port 8002

# Conformance
py conformance/runner.py --all    # run all JSON fixture suites (subprocess)
py conformance/verify.py         # run 52 in-process tests
py conformance/smoke.py          # 21-test stdlib smoke suite
py conformance/runner_js.py --all # JS conformance parity runner

# Shadow Thirst
shadowthirst check examples/promote.shadowthirst
shadowthirst visualize examples/promote.shadowthirst

# TARL tests
pytest tarl/tests/test_tarl_integration.py -v

# Iron Path execution
python -m governance.iron_path sovereign-demo.yaml

# TARL OS bridge
python tarl_os/bridge.py

```

13.4 Adding a Keyword

Add the new keyword name and `TokenType` variant to `src/utf/thirsty_lang/token.py` (both the `TokenType` enum and the `KEYWORDS` dict).

Add parsing logic to `src/utf/thirsty_lang/parser.py`.

Add AST node to `src/utf/thirsty_lang/ast.py`.

Add evaluation logic to `src/utf/thirsty_lang/interpreter.py`.

Add type-checking logic to `src/utf/thirsty_lang/checker.py`.

Add error code to `src/utf/thirsty_lang/diagnostics.py` (`ERROR_CODES` registry).

Add conformance test fixture entry in `conformance/`.

Add tests to `src/utf/tests/test_thirsty_lang.py`.

13.5 Adding a TARL Policy

Create a `.tarl` file with policy `Name { when ... => VERDICT; }` syntax.

Load via `tarl.core.parse_policy(text)`.

Evaluate via `tarl.core.evaluate(policy, context)`.

For production use, wrap in `TarlPolicy` and pass to `TarlRuntime`.

13.6 Adding a Shadow Thirst Analyzer

Create `src/utf/shadow_thirst/plugins/my_analyzer.py`.

Define `analyze_plugin(module: ShadowModule) -> list[AnalysisResult]`.

The plugin loader will automatically discover and run it at promote time.

13.7 Known Dependencies

- Python 3.11+ (required for `tomllib` stdlib support)
- Node.js 14+ (for JS conformance runner `conformance/runner_js.py`)
- `pytest` for TARL tests
- `pyyaml` for Iron Path pipeline loading and `thirst::yaml` stdlib
- `tomli` (optional fallback for `thirst::toml` on Python < 3.11)
- `pyyaml` (`pip install pyyaml`) for `thirst::yaml` namespace
- `reportlab` for PDF generation (separate tool)
- Standard library only for `src/utf` core — no third-party runtime deps

13.8 Package Manager

Source: `src/utf/thirsty_lang/package_manager.py`

Thirsty-Lang ships a native package manager (`thirsty add/audit/lock`). Package metadata is declared in `thirsty.toml` (preferred) or `thirsty.json` (legacy). The lock file includes a SHA-256 hash per dependency for integrity verification. `thirsty audit` checks all installed package hashes against the lock file.

14. Current Implementation Status

Phase Summary

Phase	Date	Key Deliverables
Phase 1	2026-05-11	Core v1.0 freeze, module/mode header, 8 formal spec docs, STABILITY.md, 19 tests pass
Phase 2	2026-05-12	stdlib 3→15 namespaces, PuritySpringAnalyzer implemented, package manager, 150+ conformance tests, pyproject v1.0.0
Phase 3A	2026-05-12	enum/struct/interface/requires keywords, Result[T,E]/Governed[T] types, governance annotations, AST-based formatter, diagnostics registry
Phase 3B	2026-05-12	Runtime governance enforcement, auto TARL policy generation (thirsty govern), governance_report, governance_proof in manifest
Phase 4	2026-05-12	WASM targets (wasm-pyodide, js), --emit-manifest with TSCG-B frame, thirsty run --demo, JS conformance runner, showcase app
Phase 5	2026-05-12	LSP server, Triumvirate (full spec+impl), PSIA (full spec+impl), TSCG-B StreamDecoder, JS/Python parity analysis, LLVM backend, green threads/async, central registry, docs site

Implemented (Evidence in Source Code)

Component	Status	Evidence
Thirsty-Lang lexer	Implemented	src/utf/thirsty_lang/lexer.py
Thirsty-Lang parser (module header, governed fns)	Implemented	src/utf/thirsty_lang/parser.py
Thirsty-Lang AST (incl. EnumDecl, StructDecl, InterfaceDecl, GovernedFunctionDecl)	Implemented	src/utf/thirsty_lang/ast.py
Thirsty-Lang checker (full Phase 3 type system)	Implemented	src/utf/thirsty_lang/checker.py
Thirsty-Lang interpreter (governance_context, mode dispatch)	Implemented	src/utf/thirsty_lang/interpreter.py
Thirsty-Lang CLI (run/repl/fmt/new/build/govern/add/audit/lock)	Implemented	src/utf/thirsty_lang/cli.py

Component	Status	Evidence
Token definitions (all keywords incl. Phase 1–3 additions)	Implemented	src/utf/thirsty_lang/token.py
Type system (Phase 1–3: all generic + governance types)	Implemented	src/utf/thirsty_lang/typesys.py
Module system (15 namespaces)	Implemented	src/utf/thirsty_lang/module_system.py
Package manager (thirsty.toml, SHA-256 lock)	Implemented	src/utf/thirsty_lang/package_manager.py
Diagnostics registry (ERROR_CODES, all E-codes)	Implemented	src/utf/thirsty_lang/diagnostics.py
AST-based formatter	Implemented	src/utf/thirsty_lang/formatter.py
WASM/Pyodide build target	Implemented	src/utf/thirsty_lang/cli.py (--target wasm-pyodide)
JS build target	Implemented	src/utf/thirsty_lang/cli.py (--target js)
Governance manifest (--emit-manifest)	Implemented	Includes TSCG-B frame, policy_hash, capability_manifest, governance_proof
Auto TARL generation (thirsty govern)	Implemented	src/utf/thirsty_lang/cli.py emit_auto_tarl()
Conformance suite (150+ tests, subprocess + in-process + smoke)	Implemented	conformance/runner.py, verify.py, smoke.py, JSON fixtures
JS conformance runner	Implemented	conformance/runner_js.py
TARL policy parser	Implemented	src/utf/tarl/core.py
TARL policy evaluator (default DENY)	Implemented	src/utf/tarl/core.py
TarLRuntime (cached, parallel)	Implemented	tarl/runtime.py
TarLPolicy / TarLVerdict / TarLDecision	Implemented	tarl/spec.py, tarl/policy.py
TARL full compiler frontend	Implemented	tarl/compiler/__init__.py
TARL bytecode VM	Implemented	tarl/runtime/__init__.py
TARL config / diagnostics / stdlib / FFI / modules / adapters	Implemented	tarl/ subdirs
Shadow Thirst parser + 6 analyzers (PuritySpringAnalyzer now real, not stub)	Implemented	src/utf/shadow_thirst/core.py
Shadow Thirst promote/reject + visualizer + plugin system	Implemented	src/utf/shadow_thirst/core.py
TSCG parser/encoder/checksum/validation	Implemented	src/utf/tscg/core.py
TSCG-B binary codec (CRC32 + SHA-256)	Implemented	src/utf/tscg_b/core.py
TARL OS kernel, security, config, AI, API, observability, IO, deployment, UI	Implemented	tarl_os/*.thirsty
Thirsty's Security Constitution (5 rules, armor lock)	Implemented	tarl_os/security/thirstys_constitution.thirsty
Thirsty's Enforcement Gateway (fail-closed)	Implemented	tarl_os/security/thirstys_enforcement_gateway.thirsty
Iron Path Executor + Sovereign Runtime	Implemented	governance/iron_path.py, governance/sovereign_runtime.py

Component	Status	Evidence
build.tarl	Implemented	build.tarl
Showcase: governed_agent_runner	Implemented	src/utf/examples/showcase/governed_agent_runner/main.thirsty
Stability contract	Implemented	STABILITY.md
8 formal spec documents	Implemented	src/utf/docs/THIRSTY_LANG_SPEC.md, THIRSTY_EBNF_GRAMMAR.md, THIRSTY_TYPE_SYSTEM.md, THIRSTY_RUNTIME_SEMANTICS.md, THIRSTY_MODULE_SYSTEM.md, THIRSTY_STANDARD_LIBRARY.md, THIRSTY_ERROR_CODES.md, THIRSTY_CONFORMANCE.md

Phase 5 New Implementations

Component	Status	Evidence
LSP server (thirsty lsp)	Implemented	src/utf/thirsty_lang/lsp_server.py (420 lines); stdio + TCP modes
Triumvirate governance (full spec + impl)	Implemented	governance/triumvirate_server.py; TRIUMVIRATE_SPEC.md; 3 pillars, FourLaws, Chimera bridge
PSIA — Plane Separation / Isolation Architecture	Implemented	src/psia/ (full module, 7 stages, Merkle tree, Ed25519); PSIA_SPEC.md; acronym confirmed
TSCG-B StreamDecoder	Implemented	src/utf/tscg_b/core.py; buffered multi-frame, magic resync, pending_bytes
JS/Python parity analysis	Implemented	src/utf/docs/JS_PYTHON_PARITY.md; verdict: 0/200 conformance; root cause: no AST pipeline in JS
LLVM backend	Implemented	src/utf/tarl/compiler/frontend.py, llvm_backend.py; targets: llvm-ir/obj/exe/asm/jit
Green threads / async (cascade glass)	Implemented	interpreter.py; TaskValue → concurrent.futures.Future; ThreadPoolExecutor; await blocks on future.result()
Central package registry	Implemented	src/utf/thirsty_lang/registry_server.py (FastAPI, port 9000); publish/search/yank/download endpoints; THIRSTY_REGISTRY_URL env var routing
Docs site generator (thirsty docs)	Implemented	src/utf/thirsty_lang/docs_generator.py (508 lines); static HTML; sidebar TOC; search index; syntax highlighting; responsive dark CSS

Previously Partial — Now Fully Implemented

Item	Prior Status	Current Status
Triumvirate governance	Stub (<code>triumvirate_server.py</code> present, unread)	Implemented — 3 pillars (Galahad/Cerberus/CodexDeus), FourLaws, Chimera bridge, full FastAPI service
PSIA planes	Directory existed, no docs	Implemented — 7-stage waterfall, Merkle tree with inclusion proofs, Ed25519 anchor, FastAPI gateway; acronym confirmed
LSP server	Port 9898 configured only	Implemented — <code>lsp_server.py</code> , <code>thirsty lsp</code> CLI subcommand
TSCG-B streaming decoder	Single-frame only	Implemented — <code>StreamDecoder</code> class with magic resync
TARL LLVM backend	Planned	Implemented — <code>ThirstyFrontend</code> + <code>LLVMBackend</code> via <code>llvmlite</code>
Green threads / async	Synchronous wrapper only	Implemented — true concurrent <code>Future</code> -based <code>TaskValue</code>
Central package registry	URL referenced only	Implemented — local FastAPI registry server, port 9000
UTF docs site	Referenced, not found	Implemented — <code>docs_generator.py</code> , <code>thirsty docs</code> command
JS/Python parity	Unclarified	Documented — 0/200 conformance; JS needs full rewrite (Option A recommended)

Still Open

Component	Status	Notes
JS implementation rewrite	Open	Parity analysis complete; rewrite estimated 4–6 weeks
TARL OS v3.0 vs v2.0 version conflict	Open	README says v2.0, Implementation Report says v3.0
TARL Source-level Debugger	Partial	Port 9899 configured; implementation unconfirmed
Ed25519 key rotation / HSM support in PSIA	Future phase	Manual env var only; no rotation protocol
Persistent Triumvirate audit log	Future phase	In-memory ring buffer only (max 1000 entries)
T-SECA/GHOST threshold cryptography	Future phase	Not implemented; listed in PSIA spec as known limitation

15. Roadmap

Based on repository evidence and stated roadmaps, updated to reflect Phases 1–4 completion (2026-05-12).

Completed (Phases 1–4)

Item	Completed	Notes	
Formal grammar specification (EBNF)	✓ Phase 1	THIRSTY_EBNF_GRAMMAR.md	
Formal language spec	✓ Phase 1	THIRSTY_LANG_SPEC.md — replaces one-page stub	
Type system spec	✓ Phase 1	THIRSTY_TYPE_SYSTEM.md	
Module system spec	✓ Phase 1	THIRSTY_MODULE_SYSTEM.md	
Runtime semantics spec	✓ Phase 1	THIRSTY_RUNTIME_SEMANTICS.md	
Error codes registry	✓ Phase 1	THIRSTY_ERROR_CODES.md	
Conformance spec	✓ Phase 1	THIRSTY_CONFORMANCE.md	
Stability contract	✓ Phase 1	STABILITY.md (Core 1.x semver guarantee)	
Module/mode header	✓ Phase 1	module name / `mode core\`	governed`
Stdlib 15 namespaces	✓ Phase 2	fs, path, json, http, env, process, log, test, collections, net, sqlite, yaml, toml + crypto + reservoir	
PuritySpringAnalyzer (real impl)	✓ Phase 2	Rejects invariant blocks with I/O or non-determinism	
Package manager	✓ Phase 2	thirsty.toml, SHA-256 lock, thirsty add/audit/lock	
Conformance test suite (150+ tests)	✓ Phase 2	subprocess runner, in-process verify, smoke suite	
enum / struct / interface types	✓ Phase 3A	Full parser, checker, interpreter support	
Result[T,E] / Governed[T] types	✓ Phase 3A	Implemented in typesys.py	
Governance annotations (requires)	✓ Phase 3A	AuthorityClass, AuditTrail, HumanAppealWindow	
AST-based formatter	✓ Phase 3A	thirsty fmt now uses Formatter class	
Runtime governance enforcement	✓ Phase 3B	_enforce_governance() in interpreter	
Auto TARL policy generation	✓ Phase 3B	thirsty govern → .auto.tarl file	

Item	Completed	Notes	
Governance manifest (<code>--emit-manifest</code>)	✓ Phase 4	TSCG-B frame, <code>policy_hash</code> , <code>capability_manifest</code> , <code>governance_proof</code>	
WASM/Pyodide build target	✓ Phase 4	Self-contained browser HTML, runs without a server	
JS build target	✓ Phase 4	Emits <code>dist/<module>.js</code>	
JS conformance runner	✓ Phase 4	<code>conformance/runner_js.py</code> (Node + JS CLI)	
Showcase app	✓ Phase 4	<code>governed_agent_runner/main.thirsty</code>	

Phase 5 — All 9 Deferred Items Completed (2026-05-12)

Item	Status
LSP server (<code>thirsty lsp</code> , stdio + TCP)	✓ Implemented
Triumvirate spec + implementation (3 pillars, FourLaws, Chimera)	✓ Implemented
PSIA spec + full module (7 stages, Merkle, Ed25519, FastAPI)	✓ Implemented
TSCG-B StreamDecoder with magic resync	✓ Implemented
JS/Python parity analysis (0/200 conformance documented)	✓ Documented
LLVM backend (IR/obj/exe/asm/JIT via llvmlite)	✓ Implemented
Green threads / async (<code>cascade glass</code> → <code>ThreadPoolExecutor</code>)	✓ Implemented
Central registry server (FastAPI, port 9000, <code>publish/search/yank</code>)	✓ Implemented
Docs site generator (<code>thirsty docs</code> , static HTML + search)	✓ Implemented

Near-Term (Open)

- JS implementation rewrite** — Option A (full AST-based rewrite, 4–6 weeks) to reach 200/200 conformance. Option B (Python wrapper) available for expedience.
- TARL Source-level Debugger** — Port 9899 configured; actual implementation unconfirmed.
- Persistent Triumvirate audit log** — In-memory ring buffer only; SQLite or append-only file is a future phase.
- Ed25519 key rotation / HSM** — PSIA anchor loads key from env var; no rotation protocol yet.

Medium-Term

- Thirst of Gods stable spec** — `THIRST_OF_GODS_SPEC.md` exists but is a minimal sketch.
- Governance certification program** — Formal process for certifying governance-compliant UTF implementations.
- T-SECA/GHOST threshold cryptography** — Listed in PSIA spec as a future phase (GF(257) threshold cryptography).

BFT consensus for governance mutations — Stage 4 uses Triumvirate majority rule; full BFT is future work.

16. Glossary

Term	Definition	
admissibility	The property of an action or state transition that makes it eligible to execute under current governance policy. An inadmissible action is blocked by T.A.R.L.	
Cerberus (Triumvirate pillar)	The security and containment pillar of the Triumvirate. Checks 20 threat patterns; DENY on match (confidence 0.98). Distinct from the Cerberus threat-detection agent in <code>src/cerberus/</code> .	
Chimera bridge	Deception-perimeter integration. The Triumvirate receives honeypot and canary-token intelligence from Chimera via <code>/chimera/verdict</code> and <code>/chimera/canary</code> endpoints.	
CodexDeus (Triumvirate pillar)	The constitutional law pillar of the Triumvirate. Enforces FourLaws; DENY on 12 constitutional violation patterns (confidence 0.99); ESCALATE on all mutate actions.	
AuthorityClass	Hierarchical authority level from AC1 (minimal) to AC5 (maximum). Declared in <code>requires</code> clauses on governed functions. Injected at runtime via <code>thirsty run --authority ACN</code> .	
AuditTrail.Immutable	Governance annotation. Marks that a governed function's invocation must produce an immutable audit record.	
auto.tarl	Policy file auto-generated by <code>thirsty govern</code> from <code>requires</code> clause annotations in a governed program.	
anchor (ANC)	TSCG symbol. A cryptographic state binding — anchors a canonical commit to a specific context hash.	
armor	Thirsty-Lang security keyword. Marks a value or data structure as immutable after assignment.	
audit span	A unique identifier attached to an agent action that allows it to be traced through the audit log. Required by Constitutional Rule 003.	
canonical plane	The authoritative execution plane in Shadow Thirst. Only values that have passed shadow simulation and invariant checks can be written here.	
capability authorization (CAP)	TSCG symbol. Represents a capability check that must pass before execution.	

Term	Definition	
canonical commit (COM)	TSCG symbol. Marks the successful commitment of a mutation to canonical state.	
Cerberus	Project-AI threat detection agent. Detects attack patterns and feeds them to T.A.R.L. for defensive compilation.	
checker	The semantic analysis phase of the Thirsty-Lang compiler. Performs type checking and scope resolution.	
Codex Deus Maximus	Project-AI's orchestration agent for permanent security feature integration. Works with Cerberus and T.A.R.L.	
cognition proposal (COG)	TSCG symbol. Represents a proposed cognitive/decision action entering the governance pipeline.	
constitutional rules	Five immutable rules in Thirsty's Security Constitution that govern all operations in TARL OS.	
Default = DENY	The fundamental policy posture of Project-AI. If no governance rule explicitly allows an action, it is denied.	
delta non-terminal (DNT)	TSCG symbol. Represents a mutation proposal — a proposed change to state.	
detect	Thirsty-Lang security keyword. Declares a threat detection block.	
determinism	A property required of the shadow plane in Shadow Thirst. The shadow block must produce the same result for the same input, always.	
drink	Thirsty-Lang keyword. Declares a variable (binds data into existence).	
enforcement gateway	Thirsty's Enforcement Gateway — the truth-defining layer that makes <code>allowed=False</code> mean execution is impossible.	
escalate	T.A.R.L. verdict. Blocks execution but routes to human reviewer for override.	
fail closed	Security posture where system failure results in blocking all operations rather than allowing them. Default behavior when the enforcement gateway is offline.	
fountain	Thirsty-Lang keyword. Declares a class (object blueprint).	
glass	Thirsty-Lang keyword. Declares a function (a contained computation vessel).	
governance	The system of policies, rules, and enforcement mechanisms that determine what actions are admissible. In Project-AI, governance precedes execution.	

Term	Definition	
hydrated	Thirsty-Lang keyword. The else branch — the "hydrated" alternative when the condition is not met.	
invariant	In Shadow Thirst, the gate section between shadow and canonical. Must evaluate cleanly before the canonical write executes.	
invariant engine (INV)	TSCG symbol. Represents the invariant checking phase.	
HumanAppealWindow[Nd]	Governance annotation. Marks that decisions from a governed function are appealable for N days.	
PSIA	Plane Separation / Isolation Architecture. A 7-stage, 6-plane defense pipeline in <code>src/psia/</code> . Ingestion → Schema → Classification → Shadow → Governance → Canonical → Seal. Gateway on port 8002.	
PreScreenGate	Fast O(1) absolute prohibition check that runs before Stage 0 of the PSIA pipeline. Matches against 10 hard-prohibited patterns.	
FourLaws	Four hard-coded constitutional constants of the Triumvirate. Immutable; supersede all other rules. 1: no harm to humans. 2: obey humans unless it conflicts with 1. 3: protect constitutional integrity. 4: act transparently.	
Galahad (Triumvirate pillar)	The ethics and human dignity pillar. Checks 13 harm patterns; ESCALATE on high-risk mutations; ALLOW otherwise (confidence 0.90).	
GovernanceDecision	Data model returned by <code>POST /intent</code> to the Triumvirate. Fields: <code>final_verdict</code> , <code>votes</code> (3 <code>TriumvirateVotes</code>), <code>audit_id</code> , <code>consensus</code> , <code>metadata</code> .	
Governed[T]	Return type for governance-annotated functions. Signals that the value came from a governance-checked execution path.	
governance manifest	JSON file produced by <code>thirsty build --emit-manifest</code> . Contains TSCG-B frame (base64), <code>policy_hash</code> , <code>capability_manifest</code> , and <code>governance_proof</code> with <code>iron_path_run_id</code> .	
Iron Path	The sovereign execution loop in Project-AI. Runs a complete governance-verified pipeline with full cryptographic audit trail.	
IntentRequest	Payload submitted to the Triumvirate: <code>actor</code> , <code>action</code> , <code>target</code> , <code>context</code> , <code>origin</code> , <code>risk_level</code> , <code>timestamp</code> .	

Term	Definition	
Merkle tree (PSIA)	Proper Merkle tree in <code>src/psia/crypto/merkle.py</code> . SHA-256 leaf hashing; paired internal nodes; odd-level duplication; inclusion proofs via <code>tree.proof(index)</code> ; verification via <code>MerkleTree.verify()</code> .	
mode core	Default Thirsty-Lang execution mode. Interpreter only; no TARL/Shadow/TSCG governance gates.	
mode governed	Thirsty-Lang execution mode that activates all governance gates. Functions with <code>requires</code> clauses enforce authority and audit requirements at call time.	
module header	Optional first two lines of a <code>.thirsty</code> file: <code>module <name></code> and <code>mode core</code>	<code>governed`</code> . Sets the execution mode for the entire file.
JIT	Just-In-Time compilation. The TARL VM compiles hot paths (>100 executions) to native code.	
morph	Thirsty-Lang security keyword. Activates dynamic code transformation/obfuscation in response to detected attack patterns.	
mutation	In Shadow Thirst, a proposed change to canonical state that must pass the shadow → invariant → canonical pipeline before being committed.	
paranoid mode	A defense posture in Thirsty-Lang security keywords that applies maximum security hardening to a component.	
parched	Thirsty-Lang keyword/state. Represents failure, error, or the absence of a needed resource.	
pipeline operator (->)	TSCG syntax. Chains governance symbols in sequence. Also available in Thirsty-Lang as <code>`</code>	<code>>`</code> .
plane isolation	The architectural separation between the shadow plane (simulation) and the canonical plane (authoritative state). Enforced by <code>PlanelIsolationAnalyzer</code> .	
policy	A Thirsty-Lang/T.A.R.L. construct that defines a named set of when-rules mapping conditions to verdicts.	
pour	Thirsty-Lang keyword. Outputs a value (the output "pours" out of the program).	
promote	Shadow Thirst operation. Commits a mutation to canonical state after all analyzers pass.	
proof-carrying execution	The Iron Path concept: every stage carries cryptographic proof (SHA-256, role signature, policy binding) of its governance compliance.	

Term	Definition	
PSIA	Plane Separation / Isolation Architecture. Referenced directory in <code>src/psia/</code> . Full acronym and content: undocumented / needs confirmation.	
quenched	Thirsty-Lang keyword/type. The satisfied/success state. Also the option type <code>Quenched[T]</code> wrapping optional values.	
quorum (QRM)	TSCG symbol. Represents a quorum decision — multiple parties must agree before the expression resolves.	
refill	Thirsty-Lang keyword. Loop construct (the loop "refills" until the condition is no longer met).	
reflex containment (RFX)	TSCG symbol. Represents a reflex/containment action — used to contain or limit propagation.	
reject	Shadow Thirst operation. Discards a mutation because critical analyzers failed. Canonical state is unchanged.	
replay hash	A SHA-256 digest of a Shadow Thirst mutation's structure that makes the promotion decision reproducible and auditable.	
reservoir	Thirsty-Lang keyword. A named mutable data store.	
SafeExpr	The sandboxed expression evaluator in <code>src/utf/tarl/core.py</code> that only allows safe Python AST node types during T.A.R.L. rule evaluation.	
sanitize	Thirsty-Lang security keyword. Validates and cleans a value before use. Applied to every external input in TARL OS.	
shadow plane	The simulation-only zone in Shadow Thirst. Can read but not write canonical state. Must be deterministic.	
Shadow Thirst	The mutation simulation and safety validation layer of the UTF. Implements <code>shadow → invariant → canonical</code> separation.	
shield	Thirsty-Lang security keyword. Wraps a component in a security enforcement zone.	
sip	Thirsty-Lang keyword. Reads input from the user or environment.	
sovereign runtime	The cryptographic execution layer in Project-AI that signs artifacts, maintains the audit trail, and verifies pipeline integrity.	

Term	Definition	
TARL	T.A.R.L. — Thirsty's Active Resistance Language (primary) or Trust and Authorization Runtime Layer (secondary, alternative framing). See Section 5.1 for conflict documentation.	
TARL OS	God Tier AI Operating System. All subsystems written in Thirsty-Lang. Provides kernel, security, AI orchestration, I/O, API, observability, and deployment layers.	
thirsty	Thirsty-Lang keyword. The <code>if</code> conditional — the system "thirsts" for a condition to be true.	
Thirsty-Lang	The water-metaphor source language that is the human-readable tier of the UTF. Defensive by design.	
Thirst of Gods	The advanced dialect of Thirsty-Lang, adding OOP (fountain classes), async (cascade), error handling (spillage/cleanup), and object instantiation.	
Triumvirate	Project-AI multi-party governance mechanism. <code>triumvirate_server.py</code> exists; full details undocumented.	
TSCG	Thirsty's Symbolic Constitutional Grammar. Compact deterministic symbolic encoding of governance flows using 9 core symbols.	
TSCG-B	Binary frame protocol for TSCG. Wire format with magic TSGB, CRC32 + SHA-256 integrity.	
UTF	Universal Thirsty Family. The complete umbrella of languages, encodings, runtimes, and symbolic systems in the Thirsty ecosystem.	
validated_canonical	Shadow Thirst keyword. Qualifies a mutation declaration as subject to the full shadow → invariant → canonical validation pipeline.	
well	Thirsty-Lang keyword. A read-only data store.	
StreamDecoder	TSCG-B class that accepts chunked byte streams via <code>feed()</code> , buffers partial frames, performs magic-byte resynchronization, and emits complete decoded frames via <code>frames()</code> .	
Triumvirate	Three-pillar constitutional governance service (FastAPI, port 8001). Pillars: Galahad (ethics), Cerberus (security), CodexDeus (constitutional law). A single DENY from any pillar blocks the action.	

Term	Definition	
TriumvirateVote	Per-pillar verdict struct: pillar, verdict (allow/deny/escalate), reasoning, confidence (0.0–1.0).	
TaskValue	Internal Thirsty-Lang interpreter type for cascade glass return values. Phase 5: upgraded from synchronous wrapper to concurrent .futures.Future for true concurrency.	

17. Source Map

Claim	Source File	Evidence	Confidence	Notes
T.A.R.L. = Thirsty's Active Resistance Language	tarl/README.md, tarl/docs/ARCHITECTURE.md, tarl/docs/WHITEPAPER.md	Title, abstract, and every section heading	HIGH	Consistent across 3 docs
T.A.R.L. = Trust and Authorization Runtime Layer	tarl_os/README.md (Integration section)	Integration code comment	MEDIUM	Alternative framing, one location only
UTF canonical stack has 6 members	src/utf/docs/CANONICAL_STACK.md	Numbered list	HIGH	Primary source
UTF README describes amplified bootstrap	src/utf/README.md	Full document	HIGH	Primary source
Thirsty-Lang keywords (full set)	src/utf/thirsty_lang/token.py	KEYWORDS dict + TokenType enum	HIGH	Definitive keyword list
Thirsty-Lang lexer implementation	src/utf/thirsty_lang/lexer.py	Complete Python class	HIGH	Fully implemented
Shadow Thirst spec	src/utf/docs/SHADOW_THIRST_SPEC.md	Document	MEDIUM	Brief (spec sketch only)
Shadow Thirst core implementation	src/utf/shadow_thirst/core.py	Full Python module	HIGH	Fully implemented
Shadow Thirst 6 analyzers	src/utf/shadow_thirst/core.py	analyze() function	HIGH	All 6 confirmed
Shadow Thirst promote/reject verdict	src/utf/shadow_thirst/core.py	promote() function	HIGH	Returns PROMOTE or REJECT
TSCG 9 core symbols	src/utf/tscg/core.py	CORE_SYMBOLS dict	HIGH	Definitive
TSCG parser/encoder/decoder	src/utf/tscg/core.py	Full Python module	HIGH	Fully implemented
TSCG-B binary format (magic TSGB)	src/utf/tscg_b/core.py	MAGIC = b"TSGB"	HIGH	Definitive
TSCG-B opcode table	src/utf/tscg_b/core.py	OPCODES dict	HIGH	Definitive
TSCG-B CRC32 + SHA-256	src/utf/tscg_b/core.py	pack_text() / unpack_frame()	HIGH	Verified in code
TARL policy syntax	src/utf/tarl/core.py, src/utf/examples/policy.tarl	Regex patterns + example	HIGH	Both confirmed
TARL policy default DENY	src/utf/tarl/core.py line 151	return "DENY" when no rule matches	HIGH	Code evidence
TarlRuntime caching, parallel eval	tarl/runtime.py	Class implementation	HIGH	Fully implemented
TarlVerdict (ALLOW/DENY/ESCALATE)	tarl/spec.py	TarlVerdict enum	HIGH	Definitive
TARL OS 29 subsystems, 13,600 LOC	tarl_os/TARL_OS_COMPLETE_IMPLEMENTATION_REPORT.md	Report claims	MEDIUM	Self-reported; not independently counted

Claim	Source File	Evidence	Confidence	Notes
TARL OS v2 vs v3 conflict	tarl_os/README.md (v2), tarl_os/TARL_OS_COMPLETE_IMPLEMENTATION_REPORT.md (v3)	Two documents	DOCUMENTED CONFLICT	Report is newer (2026-02-08 vs 2026-01-30)
Constitutional 5 rules + actions	tarl_os/security/thirstys_constitution.thirsty	initConstitution() function	HIGH	All 5 rules confirmed
Enforcement gateway fail-closed	tarl_os/security/thirstys_enforcement_gateway.thirsty	!enforcementActive branch	HIGH	Code evidence
Iron Path cryptographic stages	governance/iron_path.py	_execute_stage() implementation	HIGH	Fully implemented
Sovereign Runtime signing	governance/sovereign_runtime.py	File confirmed present	MEDIUM	Content not fully read
build.tarl Thirsty syntax	build.tarl	Full file content	HIGH	Thirsty-Lang build script
Scheduler 8 priority levels	tarl_os/kernel/scheduler.thirsty	PRIORITY_* drink declarations	HIGH	Confirmed
PSIA acronym expansion	None	Inferred from directory name + Shadow Thirst spec	LOW	Undocumented — needs confirmation
Triumvirate full implementation	governance/triumvirate_server.py	File exists	LOW	Content not read
Cerberus threat detection	tarl/README.md, tarl/docs/ARCHITECTURE.md	Integration section	MEDIUM	Bridge file referenced

18. Appendix

A. Keyword Index (Complete — from src/utf/thirsty_lang/token.py)

Language core: drink, pour, sip, thirsty, hydrated, thirst, quench, refill, times, glass, reservoir, well, of, flood, drip, evaporate, condense, fountain, return, parched, quenched, empty, mut

Modules: import, from, as

Security: shield, sanitize, armor, morph, detect, defend

Thirst of Gods (OOP/Async): cascade, this, new, public, private, await, spillage, cleanup, finally, error, throw

Policy (T.A.R.L.): policy, when, ALLOW, DENY, ESCALATE

Shadow Thirst: mutation, validated_canonical, invariant, shadow, canonical, promote, reject

B. TSCG Symbol Reference

Symbol	Opcode (TSCG-B)	Meaning		
COG	0x02	Cognition proposal		
DNT	0x03	Delta non-terminal / mutation proposal		
SHD	0x04	Deterministic shadow simulation		
INV	0x05	Invariant engine		
CAP	0x06	Capability authorization		
QRM	0x07	Quorum		
COM	0x08	Canonical commit		
ANC	0x09	Anchor		
RFX	0x0B	Reflex containment		
SAFE	0x0D	Safe execution marker		
->	0x10	Pipeline operator		
^	0x11	AND-combination		
\	\		0x14	OR-combination

C. TSCG-B Frame Diagram

Offset	Bytes	Field
-----	-----	-----
0	4	Magic: 0x54 0x53 0x47 0x42 ("TSGB")
4	1	Version: 0x01
5	1	Flags: 0x00

6	2	Payload length (big-endian uint16)
8	N	Encoded payload (opcode stream)
8+N	4	CRC32 of payload (big-endian uint32)
12+N	32	SHA-256 of canonical text

Total minimum frame size: 4 + 1 + 1 + 2 + 0 + 4 + 32 = 44 bytes.

D. Shadow Thirst Analysis Decision Matrix

```
For each mutation in the module:

PlaneIsolationAnalyzer:
  shadow_body writes canonical_ variable → FAIL (critical)
  shadow_body does not write canonical_ → PASS

DeterminismAnalyzer:
  shadow_body contains now/rand/random reference → FAIL (critical)
  shadow_body is deterministic → PASS

ResourceEstimator:
  estimated_cpu_ms > 1000 OR estimated_mem_mb > 256 → WARN
  within bounds → PASS

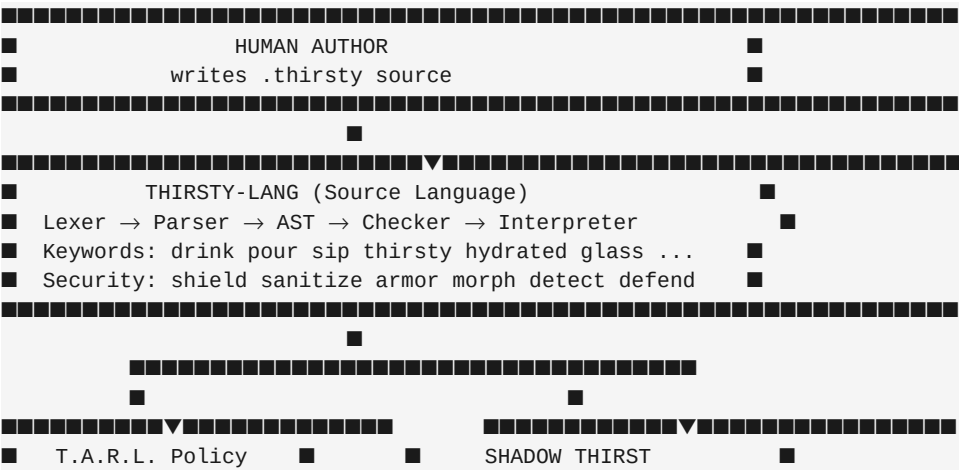
PuritySpringAnalyzer:
  invariant blocks pure → PASS (always in bootstrap model)

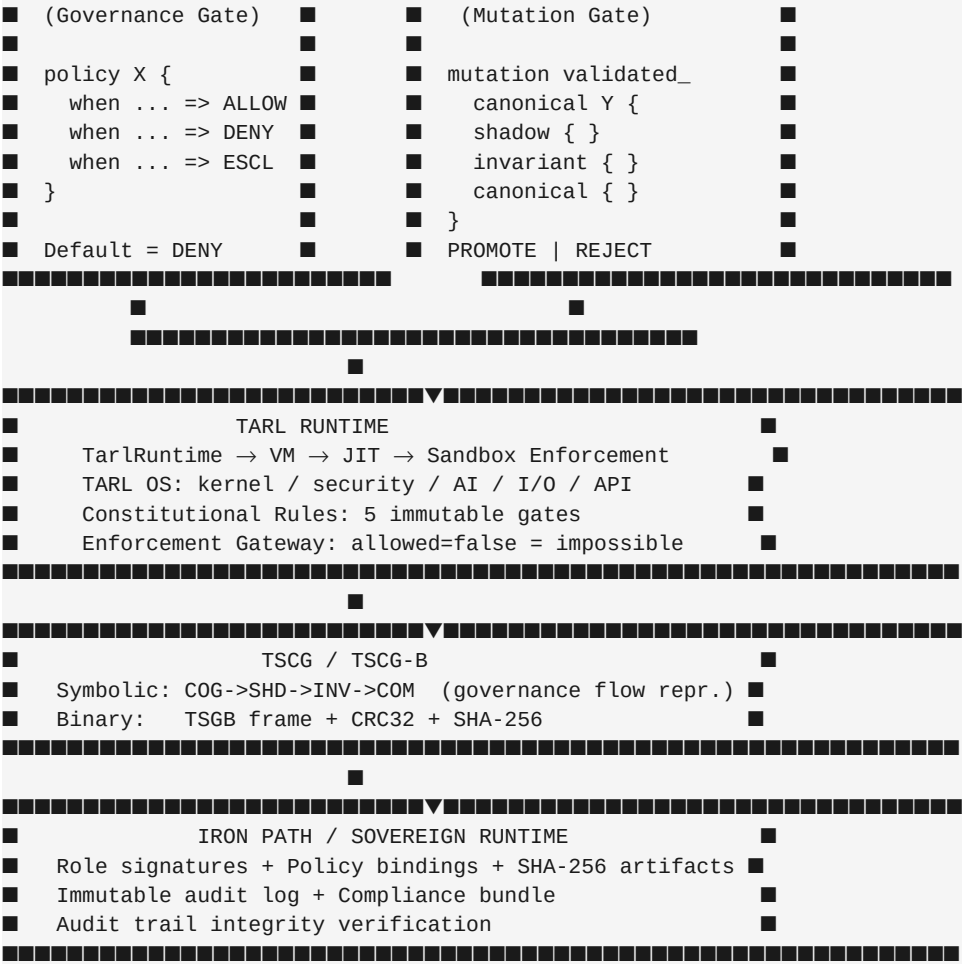
MemoryEvaporationAnalyzer:
  peak_mem_mb > 256 → WARN
  within bounds → PASS

CanonicalConvergenceAnalyzer:
  shadow writes canonical_ OR shadow is non-deterministic → FAIL (critical)
  otherwise → PASS

Promotion Decision:
  any critical FAIL → REJECT
  all critical PASS → PROMOTE (warnings noted but not blocking)
```

E. Architecture Diagram — UTF Stack





F. Unresolved Questions

- What is the exact full expansion of PSIA? The `src/psia/` directory exists but its README or index was not read.
- What does `triumvirate_server.py` implement? The quorum/multi-party governance mechanism is referenced but the file content was not inspected.
- Is the TARL LSP server (`tarl/tooling/lsp/`) fully implemented or only scaffolded? The README says "planned — infrastructure ready."
- What is the relationship between the JS `src/thirsty_lang/` implementation and the Python `src/utf/thirsty_lang/` implementation? Are they maintained in sync or is one deprecated?
- Is `registry.tarl-lang.org` live and operational?
- What does `src/thirsty_lang/src/security/threat-detector.js` implement in detail? It is referenced by TARL architecture docs but not read.
- Does the TARL OS Tier 6 (API layer) provide actual HTTP servers or only `.thirsty` declarations interpreted via the bridge?
- Is `src/utf/thirsty_lang/typesys.py` a complete type system or a placeholder?

G. Test Commands

```
# Thirsty-Lang / UTF full test suite
PYTHONPATH=src python -m unittest discover -s tests -v

# Individual test modules
PYTHONPATH=src python -m pytest src/utf/tests/test_thirsty_lang.py -v
PYTHONPATH=src python -m pytest src/utf/tests/test_tarl.py -v
PYTHONPATH=src python -m pytest src/utf/tests/test_shadow_thirst.py -v
PYTHONPATH=src python -m pytest src/utf/tests/test_tscg.py -v

# TARL runtime tests
pytest tarl/tests/test_tarl_integration.py -v
pytest tarl/tests/test_tarl_integration.py --cov=tarl --cov-report=html

# TARL OS tests
pytest tarl_os/tests/test_tarl_os.py -v

# Iron Path
python -m governance.iron_path sovereign-demo.yaml

# TARL OS bridge
python tarl_os/bridge.py
```

End of Thirsty-Lang and UTF: Universal Thirsty Family — Complete Technical, Architectural, and Governance Reference

Version 1.0 | Compiled from repository sources | 2026-05-11